Experiment No:1				
Date of Performance:	6/1/25			
Date of Submission:	13/1/25			
Program formation/ Execution/ ethical practices (06)	**Timely Submission (01)**	**Viva Answer (03)**	**Experiment Marks (10)**	**Teacher Signature with date**

Experiment No:1
"Attention is all you Need"

1.1 Aim: Study the paper "Attention is all you need and answer the questions"

1.2Course Outcome; Demonstrate the comprehensive understanding of various generative models

1.3Learning Outcome To understand the transformer architecture for Large Language model.

1.4 Related Theory:

The transformer architecture was introduced in a paper titled "Attention Is All You Need" by Vaswani et al. Transformer-based models, such as GPT-3 (Generative Pre-trained Transformer 3), have shown to be proficient in language translation, summarization, and contextual understanding.

Transformer architecture is a machine learning framework that has been used to make significant advancements in natural language processing (NLP) and other fields. The architecture is based on the following components:

- **Encoder-decoder structure**
  The encoder processes the input sequence into a vector, and the decoder converts that vector back into a sequence.
- **Self-attention layers**
  These layers capture long-range dependencies between words and are used in both the encoder and decoder blocks.
- **Multi-head attention**
  This extends the self-attention mechanism by allowing the model to attend to multiple representation subspaces in parallel.
- **Feed-forward networks**
  Each layer within the Transformer model houses a feed-forward network, which allows the model to process information in parallel.
- **Linear and softmax blocks**
  These blocks ensure that the transformer model can generate coherent outputs.

1.5 Abstract of paper:

1.6 MCQs:

Read the paper “Attention is all you need” and answer the following questions.
Choose the correct option from the following

1. What is the primary innovation introduced in the *"Attention Is All You Need"* paper?
a) Convolutional Neural Networks (CNNs)
b) Recurrent Neural Networks (RNNs)
c) The Transformer architecture
d) Long Short-Term Memory (LSTM) networks

2. In the Transformer, what replaces the recurrence typically used in sequence models?
a) Feedforward networks
b) Positional encoding and self-attention
c) Convolutional layers
d) Bidirectional layers

3. What is the purpose of positional encoding in the Transformer?
a) To add memory to the network
b) To encode the position of tokens in the sequence
c) To improve computational efficiency
d) To reduce the number of attention heads

4. What does the term "multi-head attention" refer to?
a) Using multiple sequences simultaneously
b) Running several attention mechanisms in parallel
c) Training multiple models and averaging their results
d) Employing various loss functions for attention computation

5. In the Transformer model, what is the purpose of masking in the decoder?
a) To prevent the model from attending to irrelevant input tokens
b) To ensure predictions for a position do not depend on future positions
c) To reduce overfitting during training
d) To speed up computation by skipping unnecessary operations

6. What are the key components of the self-attention mechanism in the Transformer?
a) Query, Key, and Value vectors
b) Embedding layers and softmax

c) Activation functions and dropout
d) Convolutional filters and pooling
7. The scaled dot-product attention computes attention weights using which formula?
a) $\text{softmax}(QK^T)$
b) $\text{softmax}(QK^T/\sqrt{d_k})$
c) $QK^T \cdot V$
d) $\text{softmax}(KQ^T + b)$

8. What is the significance of the scaling factor $\sqrt{d_k}$ in scaled dot-product attention?

a) It ensures the model's output size matches the input size
b) It prevents excessively large dot products, which can lead to vanishing gradients
c) It prevents excessively large dot products, which can lead to softmax saturation
d) It improves the computational speed of attention

9. Which optimizer was used in the *"Attention Is All You Need"* paper for training the Transformer?
a) SGD (Stochastic Gradient Descent)
b) Adam
c) RMSProp
d) Adagrad

10. What type of tasks was the Transformer initially designed for?
a) Image classification
b) Speech recognition
c) Machine translation
d) Time-series prediction

1.7 Conclusion:

In conclusion, the Transformer architecture has revolutionized the field of natural language processing, serving as the foundation for subsequent advancements, including BERT and GPT models. Its innovative approach has set a new standard for sequence transduction tasks and has paved the way for scalable, efficient, and high-performing deep learning models across a wide range of applications.

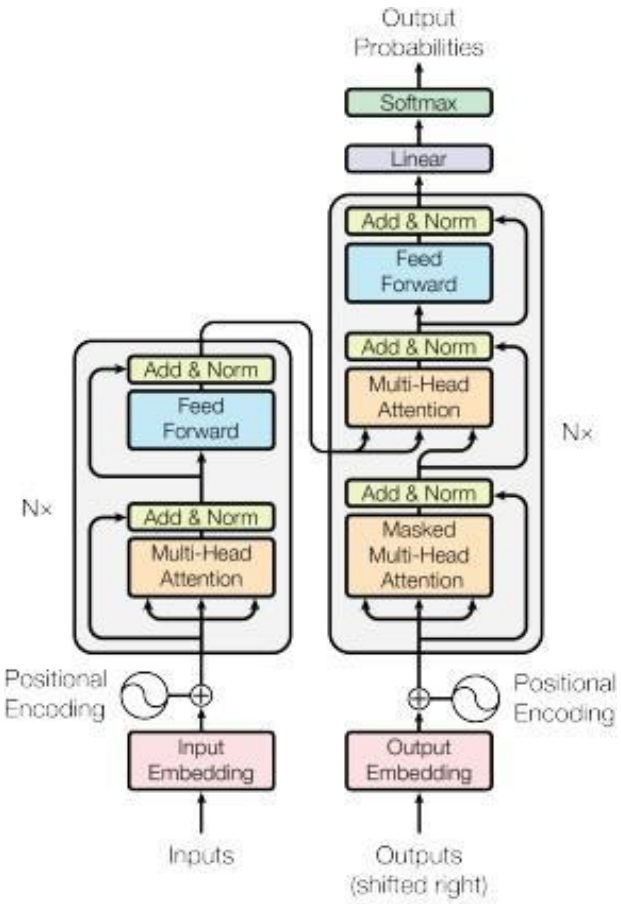
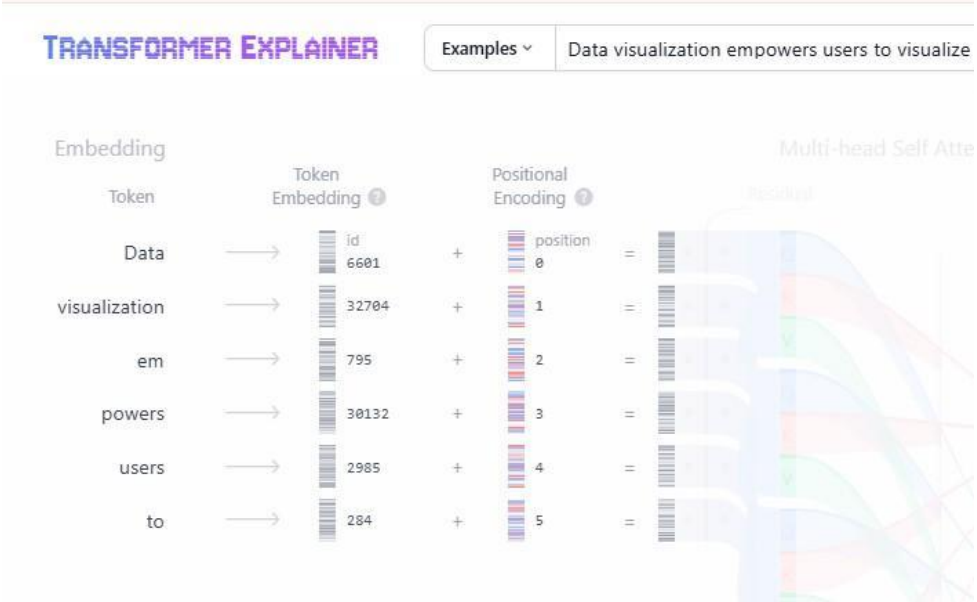
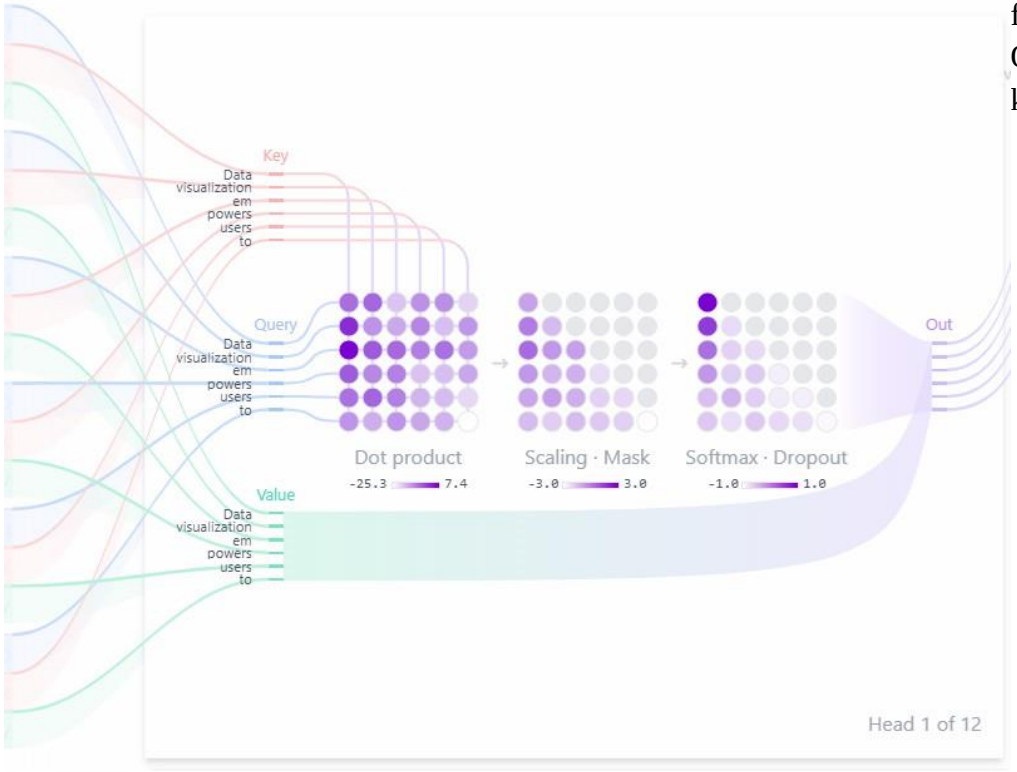


Figure 1: The Transformer - model architecture.



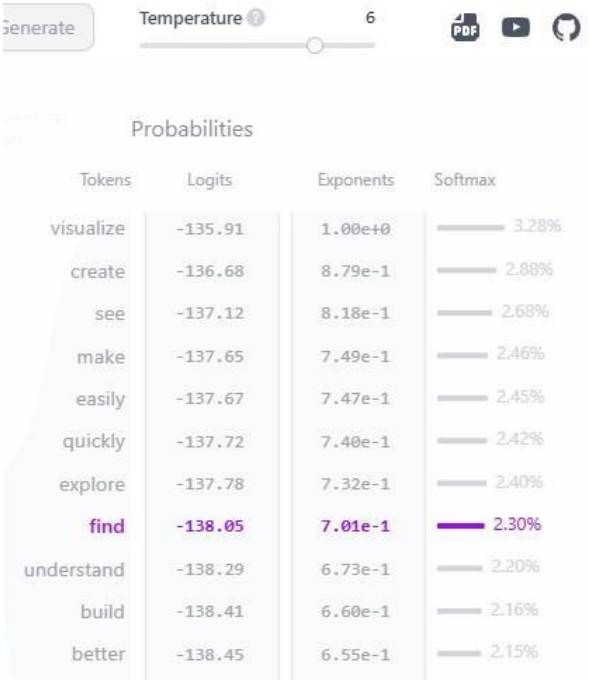
Data or prompt is tokenized with an Embedding token id or gives the number to the prompt .

And Position is assigned to every word so they cannot be jumbled or to find any mis matching condition. This is an encoding process in the encoder.

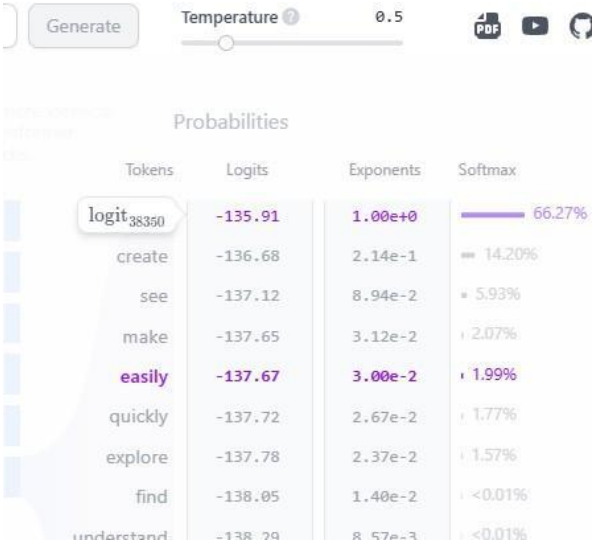


The following prompt after encoding gets multi head self attention (QKV) for m where Q= Query K= key V= value

Here the matrix multiplication (dot Product) between key and query takes place To form a Scaling mAsk which is used to check the probabilities between the words . And later on the Softmax Dropout Is done To arrange the probabilities in proper manner Mostly decreasing form. And the output Is generated.



The probabilities are been carried out And by Changing temperature we could change the probabilities outcomes. Basically it act as a filter.



Read the paper “Attention is all you need” and answer the following questions.

Choose the correct option from the following

1. What is the primary innovation introduced in the *"Attention Is All You Need"* paper?

a) Convolutional Neural Networks (CNNs)
b) Recurrent Neural Networks (RNNs)
c) The Transformer architecture
d) Long Short-Term Memory (LSTM) networks

2. In the Transformer, what replaces the recurrence typically used in sequence models?

a) Feedforward networks
b) Positional encoding and self-attention
c) Convolutional layers
d) Bidirectional layers

3. What is the purpose of positional encoding in the Transformer?

a) To add memory to the network
b) To encode the position of tokens in the sequence
c) To improve computational efficiency
d) To reduce the number of attention heads

4. What does the term "multi-head attention" refer to?

a) Using multiple sequences simultaneously
b) Running several attention mechanisms in parallel
c) Training multiple models and averaging their results
d) Employing various loss functions for attention computation

5. In the Transformer model, what is the purpose of masking in the decoder?

a) To prevent the model from attending to irrelevant input tokens
b) To ensure predictions for a position do not depend on future positions
c) To reduce overfitting during training
d) To speed up computation by skipping unnecessary operations

6. What are the key components of the self-attention mechanism in the Transformer?

a) Query, Key, and Value vectors
b) Embedding layers and softmax
c) Activation functions and dropout
d) Convolutional filters and pooling

7. The scaled dot-product attention computes attention weights using which formula?

a) $\text{softmax}(QK^T)$
b) $\text{softmax}(QK^T/\sqrt{d_k})$
c) $QK^T \cdot V$
d) $\text{softmax}(KQ^T + b)$

Option: C

8. What is the significance of the scaling factor $\sqrt{d_k}$ in scaled dot-product attention?

a) It ensures the model's output size matches the input size
b) It prevents excessively large dot products, which can lead to vanishing gradients
c) It prevents excessively large dot products, which can lead to softmax saturation
d) It improves the computational speed of attention

Option : C

9. Which optimizer was used in the *"Attention Is All You Need"* paper for training the Transformer?

a) SGD (Stochastic Gradient Descent)
b) Adam
c) RMSProp
d) Adagrad

10. What type of tasks was the Transformer initially designed for?

a) Image classification
b) Speech recognition
c) Machine translation
d) Time-series prediction

Conclusion: In conclusion, the Transformer architecture has revolutionized the field of natural language processing, serving as the foundation for subsequent advancements, including BERT and GPT models. Its innovative approach has set a new standard for sequence transduction tasks and has paved the way for scalable, efficient, and high-performing deep learning models across a wide range of applications.

Experiment No:2				
Date of Performance:	13/01/25			
Date of Submission:	20/01/25			
Program formation/ Execution/ ethical practices (06)	**Timely Submission (01)**	**Viva Answer (03)**	**Experiment Marks (10)**	**Teacher Signature with date**

Experiment No:2

1.1 Aim: Study the paper "Advancements and Applications of GenerativeAI"

1.2CourseOutcome; Demonstrate the comprehensive understanding of various generative models

1.3Learning Outcome To understand the Variational Autoencoders(VAEs) and Generative Adversarial Networks(GANs) for applications in Generative AI.

1.4 Related Theory:

Generative adversarial networks and variational autoencoders are two of the most popular approaches used for producing AI-generated content. In general, GANs tend to be more widely used for generating multimedia, while VAEs see more use in signal analysis.

1.5 Abstract of paper:

As a transformative technology, generative artificial intelligence (AI) has emerged in a variety of fields such as image synthesis, text generation, music composition and creative design with diverse applications. The purpose of this paper is to provide a comprehensive overview of recent advances in generative AI techniques. To begin with, we examine the evolution of generative models from traditional methods to state-of-the-art deep learning approaches like Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and Transformers. During the following part of the paper, we discuss generative AI and its wide-ranging applications across a wide range of sectors, such as creating realistic images, writing natural language texts, composing music, and enabling creative design tasks. Our discussion also includes potential future research and development directions along with the challenges and ethical considerations associated with generative AI. A comprehensive overview of generative AI is provided in this review for researchers, practitioners, and enthusiasts.

1.6 Presentation slides: attached in separate file

1.7 Conclusion:

There has been a surge in creativity and innovation in the field of image generation due to the advancements in generative AI models, with methodologies like Generative Adversarial Networks, Variational Autoencoders, Transformer-based models, and more driving advances in a variety of fields. Despite the remarkable achievements, there remain challenges, including mode collapse, evaluation metrics, and ethical considerations. In terms of future research directions, model robustness must be improved, contextual information must be incorporated, and interdisciplinary collaborations must be fostered. It would be helpful if feature work focused on developing robust evaluation metrics, improving model interpretability and control, and addressing ethical issues associated with the responsible deployment of generative AI systems. There is no doubt that the future of generative AI is going to unlock new frontiers of creativity, transform industries and enrich the lives of people by tackling these challenges and seizing opportunities for collaboration and innovation.

Mahavir Education Trust's

Shah & Anchor Kutchhi Engineering College

Chembur, Mumbai-400088

(Affiliated to University of Mumbai)

Advancements in Generative AI

Created By: Manohar Acharya

Roll No: 18

Date of Performance: 13/1/25

Date of Submission: 20/1/1025

Introduction Overview

SIGNIFICANCE OF GENERATIVE AI Generative AI mimics human creativity by generating new content such as images, text, audio, and even music. Unlike traditional artificial intelligence that primarily focuses on classifying data or making predictions based on predefined algorithms, generative AI employs advanced models and algorithms to create entirely new outputs, simulating a process akin to human imagination and creativity.

Generative Models Evolution

This table outlines the evolution of generative models, highlighting significant methodologies and breakthroughs that have shaped the field

GENERATIVE METHOD	DESCRIPTION KEY	BREAKTHROUGHS
Markov Chains	Stochastic models for sequences	Early generative technique
Variational Autoencoders	Learning data distribution for generation	Combines encoding/decoding approaches
Generative Adversarial Networks	Generator vs. discriminator model	Set new standards in image realism
Transformers	Self-attention for languages	Revolutionized text and NLP tasks

Applications Overview

Image Synthesis: GANs are key in image synthesis, creating realistic images and enabling detailed edits. They also allow style transformations, expanding options for digital artists

Text Generation: Transformers have transformed NLP with their ability to generate coherent text, vital for storytelling and content automation. They enhance machine translation and summarization while preserving key information

Creative Fields & Healthcare: Advanced models enhance creative industries by generating music, improving graphic design, and advancing healthcare through better imaging and personalized analytics for tailored treatments

Challenges & Ethics

TECHNICAL CHALLENGES: Issues like mode collapse in GANs significantly hinder the overall effectiveness and reliability of generative models, leading to suboptimal outputs, while the evaluation of such models continues

ETHICAL CONSIDERATIONS: Concerns surrounding deepfakes, privacy breaches, and the potential misuse of generated content, such as creating false identities or misleading information, highlight the urgent need for regulations that protect individuals and society while fostering innovation in AI technologies. to pose a complex problem due to the lack of standard metrics and benchmarks. This creates challenges for researchers and practitioners alike in assessing model performance

Conclusion Overview

IMPACT OF GENERATIVE AI Generative AI is poised to significantly transform various sectors, including healthcare, finance, and education, by enhancing efficiency and fostering innovation. However, it also introduces substantial ethical concerns and challenges that require ongoing research and collaborative efforts between technologists and policymakers to develop effective solutions.

Journal of Information Technology and Sciences

e-ISSN: 2581-849X, Vol. 10, Issue 1 (January – April, 2024) pp: (20-27)

Advancements and Applications of Generative Artificial Intelligence

Dattatray G. Takale[1*], Parikshit N. Mahalle[2], Bipin Sule[3]
[1]Assistant Professor, Department of Computer Engineering, Vishwakarma institute of information Technology, Pune, Maharashtra, India
[2]Professor, Department of AI & DS, Vishwakarma Institute of Information Technology, SPPU Pune
[3]Senior Professor, Department of Engineering, Sciences (Computer Prg) and Humanities, Vishwakarma Institute of Technology, Pune, Maharashtra, India
[*]Corresponding Author: dattatray.takale@viit.ac.in

Received Date: March 04, 2024; Published Date: March 14, 2024

Abstract
As a transformative technology, generative artificial intelligence (AI) has emerged in a variety of fields such as image synthesis, text generation, music composition and creative design with diverse applications. The purpose of this paper is to provide a comprehensive overview of recent advances in generative AI techniques. To begin with, we examine the evolution of generative models from traditional methods to state-of-the-art deep learning approaches like Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and Transformers. During the following part of the paper, we discuss generative AI and its wide-ranging applications across a wide range of sectors, such as creating realistic images, writing natural language texts, composing music, and enabling creative design tasks. Our discussion also includes potential future research and development directions along with the challenges and ethical considerations associated with generative AI. A comprehensive overview of generative AI is provided in this review for researchers, practitioners, and enthusiasts.

Keywords- Deep learning, Generative Adversarial Networks (GANs), Generative Artificial Intelligence (GAI), Transformer, Variational Autoencoders (VAEs)

INTRODUCTION

The ability of Generative Artificial Intelligence (AI) to create new content, simulate human creativity, and produce realistic output has revolutionized various fields. Generative AI, unlike traditional AI systems that concentrate on classification and prediction, aims to replicate the creative process observed in humans by understanding and replicating it [1]. From image and text generation to music composition and creative design, generative AI makes significant strides through advanced algorithms and deep learning techniques.

A major contribution to artificial intelligence and technology is the development of creative and innovative ways to use it [2]. Entertainment, healthcare, marketing, and education all benefit from the ability to generate new content autonomously. The entertainment industry, for example, employs generative AI in the creation of lifelike characters, virtual worlds, and immersive virtual reality experiences. A medical image synthesis tool, a drug discovery tool, and a personalized treatment plan can be used in healthcare [3]. A generative AI platform also enables personalized ads tailored to individual preferences in marketing and advertising. From its historical roots to the current state-of-the-art models and their varied applications, this review provides a comprehensive overview of generative AI advancements and applications. The goal of our study is to examine the evolution of generative models from early rules-based systems to sophisticated deep learning architectures and to analyse their impact on different fields. We will explore the strengths, limitations, and real-world applications of generative AI techniques, including Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and Transformer-based models [4].

As generative AI represents a paradigm shift in how we interact with technology and create content, researchers, practitioners, and enthusiasts alike must understand its importance

and potential. The purpose of this review is to inspire research, innovation, and collaboration in this rapidly evolving field by providing insights into the capabilities and challenges of generative AI [5]. Our goal is to shed light on the transformative potential of generative AI and pave the way for new advancements in artificial intelligence and beyond through a comprehensive analysis of the current landscape and prospects [6].

EVOLUTION OF GENERATIVE MODELS

The idea of generative models (Fig. 1) can be traced back to the early days of research into artificial intelligence. During those early days, the primary emphasis was on designing algorithms that were capable of producing new data points that were similar to those that were found in a certain dataset [7]. The Markov chain, which was suggested by Andrey Markov, a Russian mathematician, in the late 19th century, is considered to be one of the first instances of generative models. Markov chains are stochastic processes that describe a series of events in which the likelihood of each occurrence relies solely on the state of the event that came before it. Because of this, Markov chains are useful for creating sequences of data such as text and voice [8].

During the middle of the 20th century, the area of artificial intelligence saw considerable improvements on account of the development of rule-based expert systems and symbolic artificial intelligence [9]. The simulation of human intellect and the resolution of difficult issues were accomplished by these systems via the use of explicit programming and logical reasoning. The capacity to produce fresh material or learn from data was not one of their strengths, even though they were effective in particular fields, such as expert systems for medical diagnosis and theorem proving [10].

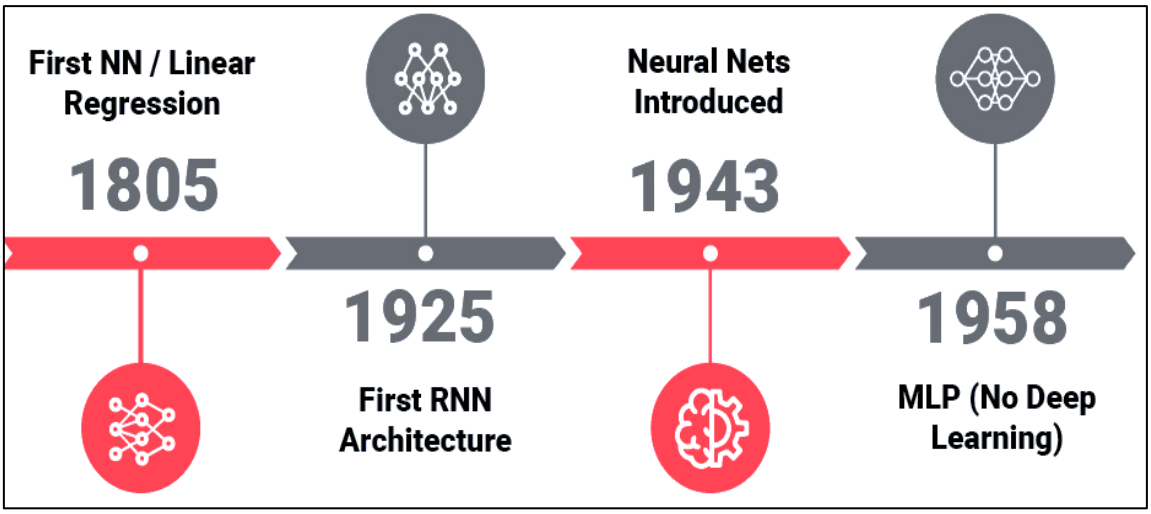


Figure 1: *Evolution of generative AI.*

In the realm of artificial intelligence, a notable breakthrough occurred when the switch from conventional methodologies to deep learning-based approaches in generative modelling were made [11]. The traditional generative models, such as Markov models and Hidden Markov Models (HMMs), have limitations due to their rudimentary designs and their inability to recognise complicated patterns in the data. Profound learning-based frameworks, then again, utilize brain networks that have various layers of connected hubs to learn progressive portrayals of information. This enables these approaches to model complex connections and provide very realistic outputs [12].

The invention of Restricted Boltzmann Machines (RBMs) by Geoffrey Hinton and his colleagues in the 2000s is considered to be one of the pioneering efforts in the field of deep learning-based generative modelling [13]. RBMs are probabilistic graphical models that learn the underlying structure of data by using a layer of visible units and a layer of hidden units together in a hierarchical construction. They were crucial in laying the groundwork for more sophisticated generative models, such as Variational Autoencoders (VAEs) and Generative

Adversarial Networks (GANs) [14], which came into being in the years that followed [15].

Overview of Key Generative AI Techniques

- ***Variational Autoencoders (VAEs):*** The VAE is a probabilistic generative model that learns to encode and decode data. The encoder networks translate input data into latent space representations, while the decoder networks reconstruct the original data from the latent space representations. To maximize the likelihood of generating input data while minimizing divergence between the latent distribution and a predefined prior distribution, variational inference techniques are employed in training VAEs [16].
- ***Generative Adversarial Networks (GANs):*** As generative models, GANs train two neural networks simultaneously: a generator network and a discriminator network, to generate data. Networks of generators generate synthetic data samples, and networks of discriminators differentiate real data from fake data. The two networks are trained adversarial, with the generator aiming to fool the discriminator and the discriminator attempting to distinguish between real and fake data [17].
- ***Transformer-Based Models:*** There have been revolutions in generative modelling in natural language processing tasks because of transformer-based models, such as the Transformer architecture introduced by Vaswani et al. in 2017. To generate coherent text that is contextually relevant to the user, transformers rely on self-attention mechanisms to capture long-range dependencies in sequential data. Several variants of the Transformer architecture have achieved state-of-the-art performance in tasks such as text generation, language translation, and document summarization. These include GPT (Generative Pretrained Transformer) and BERT (Bidirectional Encoder Representations from Transformers) [18].

APPLICATIONS OF GENERATIVE AI

Many applications of generative Artificial Intelligence (AI) techniques have been found across a wide range of domains, utilizing their ability to generate data instances similar to given datasets which is shown in Fig. 2. Several prominent generative AI applications are discussed in this section including image synthesis and manipulation with Generative Adversarial Networks (GANs), text generation and natural language processing with Transformers, music generation, design, and healthcare applications.

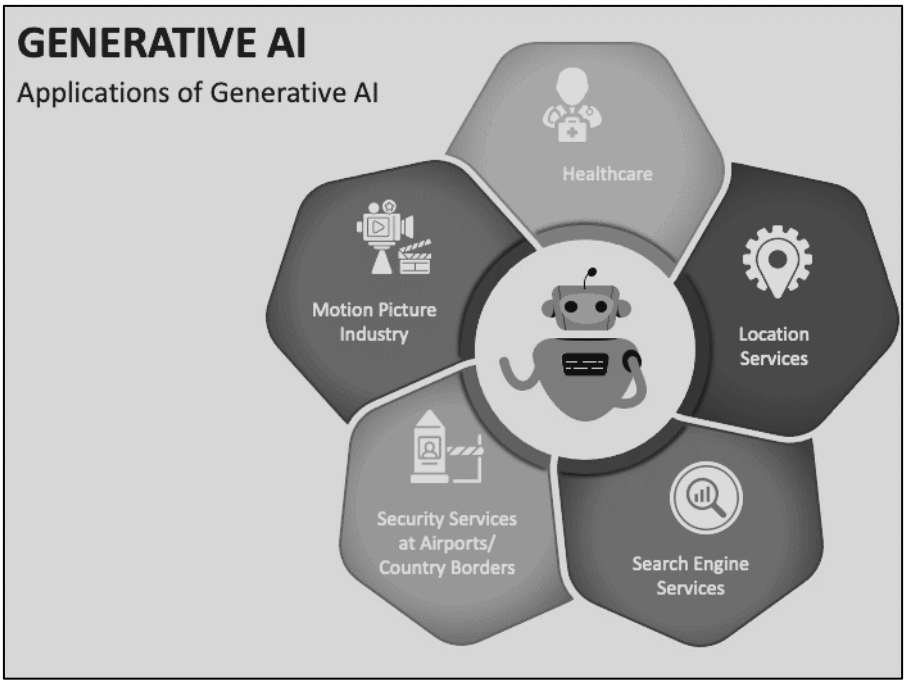


Figure 2: *Application of generative AI.*

Image Synthesis and Manipulation using GANs

There is a lot of attention being paid to Generational Adversarial Networks (GANs) due to the remarkable ability they possess to create high-quality and realistic images. In these models, a generator network is used to generate images from random noise, and a discriminator network is used to distinguish between images that are real and those that are not. GANs have been applied to a variety of domains, including:

- ***Image Generation:*** A genetic algorithm (GAN) is designed to generate realistic images of faces, landscapes, animals, and objects, amongst other things. They have been used to create artwork and create synthetic images for training datasets in various fields, as well as in generating artwork.
- ***Image Translation and Style Transfer:*** For example, GANs can be used to translate images from one domain to another, such as for converting day scenes into night scenes (for example: converting a day scene into a night scene) or for applying an artistic style to a photograph (example: applying an artistic style to a photograph)
- ***Image Editing and Manipulation:*** With training on GANs, users can edit images in a variety of ways. For example, they can change the facial expressions, the hair Color, or the background scenery of a photograph with very high precision.

Text Generation and Natural Language Processing with Transformers

Several natural language processing (NLP) tasks, including text generation, translation, summarization, and sentiment analysis, has been revolutionized by transformer-based models. As these models utilize self-attention mechanisms to capture long-range relationships in text data, they are highly effective in creating coherent and contextually relevant text. Applications for Transformers in Natural Language Processing include:

- ***Text Generation:*** Several applications have been created that use transformers to generate human-like text, such as stories, articles, dialogue, and poetry. They have been adopted in applications such as chatbots, virtual assistants, content creation platforms, and creative writing platforms.
- ***Machine Translation:*** There are several translation services and applications powered by transformers that provide high-quality, accurate and fluent translation services, allowing our customers to communicate across linguistic barriers seamlessly.
- ***Text Summarization:*** There are currently several transformations built for document summarization, news aggregation, and the curation of content that can be used to create concise summaries of long articles or documents, extracting the most important information from them while preserving their original contexts.

Music Generation and Creative Design Applications

This text summarizes several applications of generative AI to creative domains such as music generation and graphic design. These applications relied on generative models to generate engaging and engaging content in many forms:

- ***Music Generation:*** Several models can generate original music pieces, melody, harmony, and rhythms, and even mimic the style of famous composers. These models can be found in music composition software, interactive platforms for music generation, and entertainment applications.
- ***Creative Design:*** As generative AI is being used across a wide range of creative design applications, such as generating digital artwork, graphics, and animations, it allows artists and designers to discover new creative possibilities, automate repetitive tasks, and generate several designs in a short amount of time.

Other Emerging Applications

Generative AI techniques are continuously being explored and applied in emerging domains, including healthcare, gaming, and entertainment:

- ***Healthcare:*** Several tasks can be accomplished with the help of generative models in medical imaging, including reconstruction, segmentation, and anomaly detection. By generating synthetic medical

images for training deep learning models, as well as simulating medical scenarios for training healthcare professionals, these models assist in generating medical images for training.

- ***Gaming:*** As a result of the use of genetic algorithms procedural content generation techniques have become more and more common in creating environments, characters, levels and narratives for video games. These techniques enhance the realism, variety and replicability of video games, which results in immersive gaming experiences for players.
- ***Entertainment:*** Virtual reality (VR) experiences, interactive storytelling and content generation for movies, music videos and ads use generative models. With these apps, audiences get immersed and engaged in entertainment content with generative AI.

CHALLENGES AND ETHICAL CONSIDERATIONS

While generative artificial intelligence (AI) has achieved significant breakthroughs in various applications, it also poses unique ethical challenges and concerns that are unique to the field. The purpose of this chapter is to examine a few of the key challenges facing generative AI models, such as mode collapse and evaluation metrics. Additionally, it discusses some of the ethical implications of generative AI, particularly regarding privacy issues and deep fakes.

Addressing Challenges

Mode Collapse: Generated Adversarial Networks (GAN) often suffers from mode collapse, when their generator fails to capture the entire distribution of data. As a consequence, diverse and realistic samples are generated. It is necessary to design more robust training strategies to address mode collapse, such as modifying GAN architecture, incorporating regularization methods, or using alternative loss functions.

Evaluation Metrics: A generative AI model's performance is difficult to assess due to its subjective nature. The quality and diversity of generated samples are not always accurately measured by traditional metrics such as Inception Score or Frechet Inception Distance. A major research area in generative AI is the development of comprehensive evaluation metrics that take into account realism, diversity, and semantic coherence.

Ethical Implications

Deepfakes: There are significant ethical concerns about deepfakes, AI-generated synthetic media depicting individuals acting in ways they never did. In addition to face swapping and voice synthesis, generative AI also poses misinformation, defamation, and manipulation risks. Creating fake videos or audio recordings can lead to harm, political unrest, and reputation damage, as well as deceiving viewers.

Privacy Concerns: In the case of generative AI models trained on large datasets of images or text, privacy concerns are significant. As a result of these models, a highly realistic image or text of an individual can potentially be generated, thereby violating their privacy rights and damaging their reputation. Furthermore, the use of generative AI in surveillance systems or social media platforms raises serious concerns about unauthorized data collection, surveillance, and manipulation of content created by users without their permission.

Mitigating Ethical Risks

Detection and Authentication: As a means of reducing the negative effects of deepfakes, robust detection methods must be developed for identifying them. To authenticate the authenticity of media content and detect manipulated or synthetic content, researchers are experimenting with technologies such as digital watermarking, cryptographic signatures, and forensic analysis.

Regulation and Policy: A crucial role is played by policymakers and regulatory agencies when it comes to addressing the ethical challenges associated with generative AI. To prevent the misuse and abuse of AI-generated content, it is necessary to implement regulations and guidelines for the responsible use of generative AI, such as data privacy laws, transparency requirements, and content moderation policies.

Education and Awareness: Developing media literacy and critical thinking skills requires raising public awareness about deepfakes as well as other artificial intelligence-generated content

and the potential risks associated with them. By educating individuals about the capabilities and limitations of generative AI models, they can be able to discern between manipulated and authentic content and mitigate the spread of misinformation as a result.

FUTURE DIRECTIONS AND OPPORTUNITIES

As Generative Artificial Intelligence (AI) continues to evolve, several promising research directions and opportunities emerge. This section explores potential avenues for innovation, interdisciplinary collaboration, and real-world deployment of generative AI technologies.

Potential Research Directions

Improving Model Robustness: Future research efforts may concentrate on improving the resilience of generative artificial intelligence models, especially in terms of tackling typical issues such as mode collapse, training instability, and sensitivity to perturbations in input settings. Increasing the stability and dependability of generative models might be accomplished via the development of innovative training algorithms, regularisation approaches, and architectural changes.

Incorporating Contextual Information: The ability of generative AI models to generate coherent and contextually relevant content can be improved by integrating contextual information and prior knowledge into these models. Conditional generation, attention mechanisms, reinforcement learning, and other techniques can allow models to leverage context to produce more personalized and adaptive outputs.

Exploring Hybrid Approaches: It is possible to advance the capabilities of generative models by integrating different generative AI techniques, such as combining Generative Adversarial Networks (GANs) with Variational Autoencoders (VAEs) or transformer-based models with convolutional neural networks (CNNs). By combining different techniques with hybrid approaches, it would be possible to capitalize on the strengths of each technique while minimizing their respective shortcomings, resulting in more versatile and effective generative artificial intelligence systems.

Semantic Understanding and Control: The ability to manipulate and guide the generation process intuitively can be achieved by empowering generative AI models with semantic understanding and control capabilities. In this field, it might be possible for researchers to develop interpretable and controllable generative models that are capable of generating content based on the inputs of the user by specifying the desired attributes, styles, and semantics.

Opportunities for Interdisciplinary Collaboration

Human-Computer Interaction (HCI): Research in artificial intelligence and human-computer interaction can be facilitated by collaborating with experts in human-computer interaction to design user-friendly interfaces and interactive tools that can be used to generate creative content. To be effective at interacting with generative AI systems, HCI principles can be used to develop intuitive controls, feedback mechanisms, and collaborative workflows.

Cognitive Science: As a result of cognitive science insights, generative AI models can be designed to mimic the creative and cognitive abilities of humans. In the future, collaborative efforts between AI researchers and cognitive scientists may enable the creation of generative models that exhibit human-like reasoning, imagination, and problem-solving abilities, leading to new opportunities for creative AI applications.

Arts and Humanities: The arts and humanities can provide new approaches to generative AI by involving artists, designers, and scholars, and fostering interdisciplinary creativity within it. To create an enriched diversity of AI applications that explore new forms of artistic expression, cultural representation, and aesthetic innovation, AI researchers and practitioners in creative fields should collaborate to enrich the diversity of generative AI applications.

Real-World Deployment and Applications

Creative Industries: There is tremendous potential for generative AI technologies to revolutionize creative industries such as entertainment, advertising, design, and fashion in the years to come. The real-world application of generative artificial intelligence in these domains can enhance the efficiency and

creativity of creative professionals by enhancing content generation, personalized experiences, and innovative storytelling formats.

Healthcare and Education: Generative AI models can be applied in healthcare for tasks such as medical image synthesis, drug discovery, and personalized treatment planning. In education, generative AI can facilitate interactive learning experiences, personalized tutoring systems, and educational content generation, catering to individual learning styles and preferences.

Environmental Sustainability: For climate modelling and ecological conservation, generative AI techniques can be used to optimize resource utilization, design sustainable products, and simulate environmental scenarios. As a result of generative artificial intelligence, we can contribute to the development of sustainable development and environmental stewardship by generating insights and solutions to complex environmental challenges.

CONCLUSION

There has been a surge in creativity and innovation in the field of image generation due to the advancements in generative AI models, with methodologies like Generative Adversarial Networks, Variational Autoencoders, Transformer-based models, and more driving advances in a variety of fields. Despite the remarkable achievements, there remain challenges, including mode collapse, evaluation metrics, and ethical considerations. In terms of future research directions, model robustness must be improved, contextual information must be incorporated, and interdisciplinary collaborations must be fostered. It would be helpful if feature work focused on developing robust evaluation metrics, improving model interpretability and control, and addressing ethical issues associated with the responsible deployment of generative AI systems. There is no doubt that the future of generative AI is going to unlock new frontiers of creativity, transform industries and enrich the lives of people by tackling these challenges and seizing opportunities for collaboration and innovation.

REFERENCES

1. Baidoo-Anu, D., & Ansah, L. O. (2023). Education in the era of generative artificial intelligence (AI): Understanding the potential benefits of ChatGPT in promoting teaching and learning. *Journal of AI*, *7*(1), 52-62. https://doi.org/10.61969/jai.1337500
2. Harshvardhan, G. M., Gourisaria, M. K., Pandey, M., & Rautaray, S. S. (2020). A comprehensive survey and analysis of generative models in machine learning. *Computer Science Review*, *38*, 100285. https://doi.org/10.1016/j.cosrev.2020.100285
3. Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, *381*(6654), 187-192. https://www.science.org/doi/abs/10.1126/science.adh2586
4. Wach, K., Duong, C. D., Ejdys, J., Kazlauskaitė, R., Korzynski, P., Mazurek, G., ... & Ziemba, E. (2023). The dark side of generative artificial intelligence: A critical analysis of controversies and risks of ChatGPT. *Entrepreneurial Business and Economics Review*, *11*(2), 7-30. https://www.ceeol.com/search/article-detail?id=1205845
5. Castelli, M., & Manzoni, L. (2022). Generative models in artificial intelligence and their applications. *Applied Sciences*, *12*(9), 4127. https://doi.org/10.3390/app12094127
6. Bahroun, Z., Anane, C., Ahmed, V., & Zacca, A. (2023). Transforming education: A comprehensive review of generative artificial intelligence in educational settings through bibliometric and content analysis. *Sustainability*, *15*(17), 12983. https://doi.org/10.3390/su151712983
7. Kanbach, D. K., Heiduk, L., Blueher, G., Schreiter, M., & Lahmann, A. (2023). The GenAI is out of the bottle: generative artificial intelligence from a business model innovation perspective. *Review of Managerial Science*, 1-32. https://doi.org/10.1007/s11846-023-00696-z
8. Sujata, P., Takale, D. G., Tyagi, S., Bhalerao, S., Tiwari, M., & Dhanraj, J. A. (2024). New Perspectives, Challenges, and Advances in Data Fusion in Neuroimaging. *Human Cancer Diagnosis and Detection Using Exascale Computing*, *185*, 185.

https://books.google.co.in/books?hl=en&lr=&id=HNf2EAAAQBAJ&oi=fnd&pg=PA185&dq=New+Perspectives,+Challenges,+and+Advances+in+Data+Fusion+in+Neuroimaging&ots=ouXsSqYTjU&sig=yK__UPOSPOR35zi8Fzp6jkqMnWI&redir_esc=y#v=onepage&q=New%20Perspectives%2C%20Challenges%2C%20and%20Advances%20in%20Data%20Fusion%20in%20Neuroimaging&f=false

9. Santhakumar, G., Takale, D. G., Tyagi, S., Anitha, R., Tiwari, M., & Dhanraj, J. A. (2024). Analysis of Multimodality Fusion of Medical Image Segmentation Employing Deep Learning. *Human Cancer Diagnosis and Detection Using Exascale Computing*, *171*, 171. https://books.google.co.in/books?hl=en&lr=&id=HNf2EAAAQBAJ&oi=fnd&pg=PA171&dq=Analysis+of+Multimodality+Fusion+of+Medical+Image+Segmentation+Employing+Deep+Learning&ots=ouXsSqYUdQ&sig=HrHBAUwiiX_66N2KjHAqUuHxOBU&redir_esc=y#v=onepage&q=Analysis%20of%20Multimodality%20Fusion%20of%20Medical%20Image%20Segmentation%20Employing%20Deep%20Learning&f=false
10. Takale, D. G., Mahalle, P. N., Sakhare, S. R., Gawali, P. P., Deshmukh, G., Khan, V., ... & Maral, V. B. (2023, August). Analysis of Clinical Decision Support System in Healthcare Industry Using Machine Learning Approach. In *International Conference on ICT for Sustainable Development* (pp. 571-587). Singapore: Springer Nature Singapore. https://doi.org/10.1007/978-981-99-5652-4_51
11. Kadam, S. U., Dhede, V. M., Khan, V. N., Raj, A., & Takale, D. G. (2022). Machine learning methode for automatic potato disease detection. *NeuroQuantology*, *20*(16), 2102-2106.
12. Takale, D. G., Gunjal, S. D., Khan, V. N., Raj, A., & Guja, S. N. (2022). Road accident prediction model using data mining techniques. *NeuroQuantology*, *20*(16), 2904.
13. Bere, S. S., Shukla, G. P., Khan, V. N., Shah, A. M., & Takale, D. G. (2022). Analysis of students performance prediction in online courses using machine learning algorithms. *NeuroQuantology*, *20*(12), 13-19.
14. Raut, R., Borole, Y., Patil, S., Khan, V., & Takale, D. G. (2022). Skin disease classification using machine learning algorithms. *NeuroQuantology*, *20*(10), 9624-9629.
15. Kadam, S. U., Khan, V. N., Singh, A., Takale, D. G., & Galhe, D. S. (2022). Improve the performance of non-intrusive speech quality assessment using machine learning algorithms. *NeuroQuantology*, *20*(10), 12937. https://www.proquest.com/openview/6782d83267a975d8a2aaf2d5c2530d79/1?pq-origsite=gscholar&cbl=2035897
16. Takale, D. G. (2019). A review on implementing energy efficient clustering protocol for wireless sensor network. *J Emerg Technol Innov Res (JETIR)*, *6*(1), 310-315.
17. Takale, D. G. (2019). A review on QoS aware routing protocols for wireless sensor networks. *International Journal of Emerging Technologies and Innovative Research*, *6*(1), 316-320.
18. Takale, D. G. (2019). A Review on Wireless Sensor Network: its Applications and challenges. *Journal of Emerging Technologies and Innovative Research (JETIR)*, *6*(1), 222-226.

CITE THIS ARTICLE

Dattatray G. Takale et al.(2024). Advancements and Applications of Generative Artificial Intelligence, *Journal of Information Technology and Sciences*, 10(1), 20-27.

Experiment No. – 3				
Date of Performance:	20/1/25			
Date of Submission:	27/1/25			
Program Execution/ formation/ correction/ ethical practices (06)	Timely Submission (01)	Viva (03)	Experiment Total (10)	Sign with Date

1.1 Aim: To perform basic arithmetic operations using tensorflow and pytorch.

1.2 Course Outcome: Develop profiency in programming languages and libraries commonly used in AI and ML**.**

1.3 Learning Objectives:. To perform addition, subtraction, multiplication, and division of matrices using tensor flow and pytorch libraries.

1.4 Requirement:. Jupyter notebook.

1.5 Related Theory: PyTorch is defined as an open source machine learning library for Python. It is used for applications such as natural language processing. It is initially developed by Facebook artificial-intelligence research group, and Uber's Pyro software for probabilistic programming which is built on it.

PyTorch redesigns and implements Torch in Python while sharing the same core C libraries for the backend code. PyTorch developers tuned this back-end code to run Python efficiently. They also kept the GPU based hardware acceleration as well as the extensibility features that made Lua-based Torch.

TensorFlow is a software library or framework, designed by the Google team to implement machine learning and deep learning concepts in the easiest manner. It combines the computational algebra of optimization techniques for easy calculation of many mathematical expressions.

PyTorch	TensorFlow
PyTorch is closely related to the lua-based Torch framework which is actively used in Facebook.	TensorFlow is developed by Google Brain and actively used at Google.
PyTorch is relatively new compared to other competitive technologies.	TensorFlow is not new and is considered as a to-go tool by many researchers and industry professionals.
PyTorch includes everything in imperative and dynamic manner.	TensorFlow includes static and dynamic graphs as a combination.
Computation graph in PyTorch is defined during runtime.	TensorFlow do not include any run time option.
PyTorch includes deployment featured for mobile and embedded frameworks.	TensorFlow works better for embedded frameworks.

1.6 Procedure: instructions and procedure are performed in the jupyter notebook.

1.7 Program and Output: attached jupyter notebook pdf for output.

1.8Conclusion : We have successfully performed addition, subtraction, multiplication, and division of matrices using tensor flow and pytorch libraries

1.9 Questions:

- **What is PyTorch mainly used for?**

a) Game development
b) Data visualization
c) Deep learning and machine learning
d) Database management
Answer: **c) Deep learning and machine learning**

- **Which module in PyTorch is used for building neural networks?**

a) torch.nn
b) torch.optim
c) torch.utils
d) torch.data
Answer: **a) torch.nn**

 What is the default datatype for tensors in PyTorch?

a) float64
b) int32
c) float32
d) bool

Answer: **c) float32**

 Which function is used to stop tracking the gradients of a tensor in PyTorch?

a) torch.no_grad()
b) torch.autograd()
c) torch.grad_stop()
d) torch.no_gradient()

Answer: **a) torch.no_grad()**

 In PyTorch, what is the purpose of torch.utils.data.DataLoader?

a) For loading data efficiently in batches
b) For building neural network layers
c) For optimizing weights
d) For saving trained models

Answer: **a) For loading data efficiently in batches**

1. **TensorFlow is primarily developed by which organization?**
   a) Facebook
   b) Microsoft
   c) Google
   d) Apple
   Answer: **c) Google**
2. **What is the core data structure in TensorFlow?**
   a) Variable
   b) Tensor
   c) Array
   d) Matrix
   Answer: **b) Tensor**
3. **Which module in TensorFlow is used to create datasets?**
   a) tf.optim
   b) tf.nn
   c) tf.data
   d) tf.layers
   Answer: **c) tf.data**
4. **What is the purpose of tf.function in TensorFlow?**
   a) To execute code in eager mode
   b) To create a computational graph for faster execution
   c) To create a new tensor
   d) To initialize variables
   Answer: b) To create a computational graph for faster execution

5. **Which API is recommended for building deep learning models in TensorFlow?**
   a) TensorFlow Extended (TFX)
   b) TensorFlow Lite
   c) Keras
   d) TensorBoard
   Answer: c) Keras

```
Start coding or generate with AI.
```

```
import tensorflow as tf
import tensorflow as tf
tensorA = tf.constant([[1, 2], [3, 4], [5, 6]]) # Use tf.constant to create a tensor
tensorB = tf.constant([[1,-1], [2,-2], [3,-3]]) # Use tf.constant to create a tensor
result =  tf.add(tensorA, tensorB)
print ("Result:", result)
```

```
Result: tf.Tensor(
[[2 1]
 [5 2]
 [8 3]], shape=(3, 2), dtype=int32)
```

```
import tensorflow as tf
import tensorflow as tf
tensorA = tf.constant([[1, 2], [3, 4], [5, 6]]) # Use tf.constant to create a tensor
tensorB = tf.constant([[1,-1], [2,-2], [3,-3]]) # Use tf.constant to create a tensor
result =  tf.subtract(tensorA, tensorB)
print ("Result:", result)
```

```
Result: tf.Tensor(
[[0 3]
 [1 6]
 [2 9]], shape=(3, 2), dtype=int32)
```

```
import tensorflow as tf
import tensorflow as tf
tensorA = tf.constant([[1, 2], [3, 4], [5, 6]]) # Use tf.constant to create a tensor
tensorB = tf.constant([[1,-1], [2,-2], [3,-3]]) # Use tf.constant to create a tensor
result =  tf.multiply(tensorA, tensorB)
print ("Result:", result)
```

```
Result: tf.Tensor(
[[  1  -2]
 [  6  -8]
 [ 15 -18]], shape=(3, 2), dtype=int32)
```

```
import tensorflow as tf
import tensorflow as tf
tensorA = tf.constant([[1, 2], [3, 4], [5, 6]]) # Use tf.constant to create a tensor
tensorB = tf.constant([[1,-1], [2,-2], [3,-3]]) # Use tf.constant to create a tensor
result =  tf.divide(tensorA, tensorB)
print ("Result:", result)
```

```
Result: tf.Tensor(
[[ 1.         -2.        ]
 [ 1.5        -2.        ]
 [ 1.66666667 -2.        ]], shape=(3, 2), dtype=float64)
```

```
import tensorflow as tf

tensorA = tf.constant((1,2,3,4)) # Use tf.constant to create a tensor
result =  tf.square(tensorA)
print ("Result:", result)
```

```
Result: tf.Tensor([ 1  4  9 16], shape=(4,), dtype=int32)
```

```
import tensorflow as tf

tensorA = tf.constant([[1, 2], [3, 4]])
tensorB = tf.reshape(tensorA, [4, 1])
# Result: [ [1], [2], [3], [4] ]
print(tensorB)
```

```
tf.Tensor(
[[1]
 [2]
 [3]
 [4]], shape=(4, 1), dtype=int32)
```

```
import torch
```

```
x = torch.tensor([1, 2, 3])
y = torch.tensor([4, 5, 6])
result = torch.add(x, y)
print(result)
```

```
tensor([5, 7, 9])
```

```
import torch
x = torch.tensor([1, 2, 3])
y = torch.tensor([4, 5, 6])
result = torch.subtract(x, y)
print(result)
```

```
tensor([-3, -3, -3])
```

```
import torch
x = torch.tensor([1, 2, 3])
y = torch.tensor([4, 5, 6])
result = torch.multiply(x, y)
print(result)
```

```
tensor([ 4, 10, 18])
```

```
import torch
x = torch.tensor([1, 2, 3])
y = torch.tensor([4, 5, 6])
result = torch.divide(x, y)
print(result)
```

```
tensor([0.2500, 0.4000, 0.5000])
```

```
import torch
x = torch.tensor([1, 2, 3])
y = torch.tensor([4, 5, 6])
result = torch.matmul(x, y)
print(result)
```

```
tensor(32)
```

Experiment No. – 4				
Date of Performance:	27/1/25			
Date of Submission:	3/2/25			
Program Execution/ formation/ correction/ ethical practices (06)	Timely Submission (01)	Viva (03)	Experiment Total (10)	Sign with Date

4.1 Aim: To create a simple chatbot and text summarization using hugging face model.

4.2 Course Outcome: Successfully implement and train the generative ai models using appropriate software's and tools.

4.3 Learning Objectives:. Is to create a simple chat bot and text summarization application using blender bot, and text summarization application using bart model.

4.4 Requirement:. Hugging Face hug , google Collab.

4.5 Related Theory:

Hugging Face is a leading platform for natural language processing (NLP) that provides powerful tools and pre-trained models for a wide range of tasks, including text summarization, conversational agents, and more.

Text summarization, a key feature in NLP, involves condensing lengthy documents into concise summaries while preserving essential information, and Hugging Face offers models like BART (Bidirectional and Auto-Regressive Transformers) for this task.

BART is a transformer-based model designed for sequence-to-sequence tasks such as text summarization, text generation, and translation.

BlenderBot, developed by Facebook AI and available through Hugging Face, is a state-of-the-art conversational agent that combines various dialogue strategies, offering more engaging and contextually relevant interactions in real-world conversations. These models represent significant advancements in NLP, enabling users to leverage cutting-edge AI for a variety of applications.

4.6 Procedure:

1. import model from hugging face.

2. Import necessary libraries.

4.7 Program and Output: join in pdf

4.8 Conclusion: Hence we successfully created a chat bot using hugging face models.

4.9 Questions:

What is Hugging Face best known for?
a) Image Processing
b) Natural Language Processing (NLP)
c) Game Development
d) Database Management

Answer: b) Natural Language Processing (NLP)

Which of the following is a common library provided by Hugging Face?
a) TensorFlow
b) PyTorch
c) Transformers
d) Scikit-learn

Answer: c) Transformers

What file format is typically used to store pre-trained Hugging Face models?

a) .h5
b) .pt or .bin
c) .csv
d) .xml

Answer: b) .pt or .bin

Which of the following is NOT a pre-trained model available in Hugging Face?
a) BERT
b) GPT
c) VGGNet
d) RoBERTa

Answer: c) VGGNet

What is the purpose of the Hugging Face `pipeline()` function?
a) To create datasets
b) To simplify model training
c) To make it easier to use pre-trained models for common tasks
d) To visualize model architectures

Answer: c) To make it easier to use pre-trained models for common tasks

Which parameter in the Hugging Face `pipeline()` function specifies the type of task (e.g., sentiment analysis, translation)?
a) task
b) model
c) config
d) tokenizer

Answer: a) task

What is the primary advantage of using a pre-trained model from Hugging Face?
a) Reduced memory usage
b) Faster deployment without extensive training
c) Better GPU compatibility
d) Improved image processing capabilities

Answer: b) Faster deployment without extensive training

In chatbot development, what is the main role of a tokenizer in Hugging Face models?
a) To train the chatbot model
b) To convert text into input IDs that the model can understand
c) To evaluate the chatbot's responses
d) To create the conversational dataset

Answer: b) To convert text into input IDs that the model can understand

What does the `trainer.train()` function do in Hugging Face?
a) Loads the pre-trained model
b) Fine-tunes the model on a custom dataset
c) Performs inference on new data
d) Validates the model

Answer: b) Fine-tunes the model on a custom dataset

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch


tokenizer = AutoTokenizer.from_pretrained("microsoft/DialoGPT-medium")
model = AutoModelForCausalLM.from_pretrained("microsoft/DialoGPT-medium")

# Let's chat for 5 lines
for step in range(5):
    # encode the new user input, add the eos_token and return a tensor in Pytorch
    new_user_input_ids = tokenizer.encode(input(">> User:") + tokenizer.eos_token, return_tensors='pt')

    # append the new user input tokens to the chat history
    bot_input_ids = torch.cat([chat_history_ids, new_user_input_ids], dim=-1) if step > 0 else new_user_input_ids

    # generated a response while limiting the total chat history to 1000 tokens,
    chat_history_ids = model.generate(bot_input_ids, max_length=1000, pad_token_id=tokenizer.eos_token_id)

    # pretty print last ouput tokens from bot
    print("DialoGPT: {}".format(tokenizer.decode(chat_history_ids[:, bot_input_ids.shape[-1]:][0], skip_special_tokens=True)))
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secre
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%  614/614 [00:00<00:00, 41.1kB/s]
vocab.json: 100%  1.04M/1.04M [00:00<00:00, 5.09MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 3.44MB/s]
config.json: 100%  642/642 [00:00<00:00, 47.2kB/s]
pytorch_model.bin: 100%  863M/863M [00:07<00:00, 190MB/s]
generation_config.json: 100%  124/124 [00:00<00:00, 4.94kB/s]
>> User:akank
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observ
DialoGPT:
>> User:what is todays date
DialoGPT: I'm not sure, but I think it's the day after the last one.
>> User:what is todays weather
DialoGPT: I'm not sure, but I think it's the day after the last one.
>> User:today is friday
DialoGPT: I'm not sure, but I think it's the day after the last one.
>> User:toady is monday?
DialoGPT: I'm not sure, but I think it's the day after the last one.
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch


tokenizer = AutoTokenizer.from_pretrained("microsoft/DialoGPT-medium")
model = AutoModelForCausalLM.from_pretrained("microsoft/DialoGPT-medium")

# Let's chat for 5 lines
for step in range(5):
    # encode the new user input, add the eos_token and return a tensor in Pytorch
    new_user_input_ids = tokenizer.encode(input(">> User:") + tokenizer.eos_token, return_tensors='pt')

    # append the new user input tokens to the chat history
    bot_input_ids = torch.cat([chat_history_ids, new_user_input_ids], dim=-1) if step > 0 else new_user_input_ids

    # generated a response while limiting the total chat history to 1000 tokens,
    chat_history_ids = model.generate(bot_input_ids, max_length=1000, pad_token_id=tokenizer.eos_token_id)

    # pretty print last ouput tokens from bot
    print("DialoGPT: {}".format(tokenizer.decode(chat_history_ids[:, bot_input_ids.shape[-1]:][0], skip_special_tokens=True)))
```

```
>> User:toady is monday?
DialoGPT: I think it's Tuesday.
>> User:ai is what
DialoGPT: I'm not sure.
>> User:WHAT IS TOADY WEATHER
```

```
DialoGPT: I'm not sure.
>> User:
```

```
from transformers import pipeline

summarizer = pipeline("summarization", model="Falconsai/text_summarization")

ARTICLE = """
Hugging Face: Revolutionizing Natural Language Processing
Introduction
In the rapidly evolving field of Natural Language Processing (NLP), Hugging Face has emerged as a prominent and innovative force. This article
The Birth of Hugging Face
Hugging Face was founded in 2016 by Clément Delangue, Julien Chaumond, and Thomas Wolf. The name "Hugging Face" was chosen to reflect the comp
Transformative Innovations
Hugging Face is best known for its open-source contributions, particularly the "Transformers" library. This library has become the de facto st
Key Contributions:
1. **Transformers Library:** The Transformers library provides a unified interface for more than 50 pre-trained models, simplifying the develo
2. **Model Hub:** Hugging Face's Model Hub is a treasure trove of pre-trained models, making it simple for anyone to access, experiment with,
3. **Hugging Face Transformers Community:** Hugging Face has fostered a vibrant online community where developers, researchers, and AI enthus
Democratizing AI
Hugging Face's most significant impact has been the democratization of AI and NLP. Their commitment to open-source development has made power
By providing open-source models and tools, Hugging Face has empowered a diverse array of users to innovate and create their own NLP applicatio
Industry Adoption
The success and impact of Hugging Face are evident in its widespread adoption. Numerous companies and institutions, from startups to tech gia
Future Directions
Hugging Face's journey is far from over. As of my last knowledge update in September 2021, the company was actively pursuing research into eth
Conclusion
Hugging Face's story is one of transformation, collaboration, and empowerment. Their open-source contributions have reshaped the NLP landscape
"""
print(summarizer(ARTICLE, max_length=1000, min_length=30, do_sample=False))
>>> [{'summary_text': 'Hugging Face has emerged as a prominent and innovative force in NLP . From its inception to its role in democratizing A
```

Experiment No. – 5				
Date of Performance:	3/2/2025			
Date of Submission:	24/2/2025			
Program Execution/ formation/ correction/ ethical practices (06)	Timely Submission (01)	Viva (03)	Experiment Total (10)	Sign with Date

5.1 Aim: Create a translation application using langchain and groq

5.2 Course Outcome: Successfully implement and train this models using appropriate software tools and framework

5.3 Learning Objectives:. Students will be able to create an application of translation using langchain and groq
5.4 Requirement:. Google Colab ,Groq, mixtral.

5.5 Related Theory:

LangChain is an open-source framework designed to help developers build applications that integrate with large language models (LLMs) like OpenAI's GPT, Google's Gemini, or open-source models like LLaMA. It provides tools and abstractions for building context-aware, multi-step, and data-driven AI applications.

Key Components:

1. **LLMs** – Connects with models like OpenAI, Hugging Face, and others.
2. **Prompt Templates** – Helps standardize and optimize prompt structures.
3. **Memory** – Allows models to retain context across conversations.
4. **Retrieval-Augmented Generation (RAG)** – Enhances responses by retrieving relevant information from a knowledge base.
5. **Agents & Tools** – Enables decision-making workflows by calling external APIs, databases, or performing actions.
6. **Chains** – Connects multiple components to create pipelines for complex AI applications.

Common Use Cases:

- Chatbots & Virtual Assistants
- Document Search (RAG)
- Code Generation & Debugging
- Automated Data Analysis

5.6 Procedure:

Open google colab and write the code

5.7 Program and Output: A pdf of codes and output is joined in another file

5.8 Conclusion:

In this experiment, we successfully developed a translation application using **LangChain** and **Groq**, demonstrating the power of large language models in real-time text translation. By leveraging LangChain's modular framework and Groq's efficient processing capabilities, we achieved seamless language conversion with high accuracy.

5.9 Questions:

1. What is LangChain primarily used for?

a) Data Analysis
b) Building chatbots
c) Developing machine learning models
d) Building applications that use language models

Answer: d) Building applications that use language models

2. In LangChain, a "Chain" typically refers to:

a) A sequence of API calls
b) A sequence of operations or transformations applied to data
c) A set of language models interacting together
d) A set of web pages linked together

Answer: b) A sequence of operations or transformations applied to data

3. What is the role of the LLMChain in LangChain?

a) It helps manage external APIs.
b) It manages interactions with multiple language models.
c) It chains together several natural language processing (NLP) tasks.
d) It stores data from the user.

Answer: b) It manages interactions with multiple language models.

4. What kind of external data source can LangChain integrate with?

a) Local files only
b) SQL databases, APIs, and web scraping
c) Only text-based data
d) No external data sources

Answer: b) SQL databases, APIs, and web scraping

5. LangChain offers tools for:

a) Natural language understanding (NLU)
b) Text generation, summarization, and Q&A
c) Image processing
d) Video analysis

Answer: b) Text generation, summarization, and Q&A

6. What does the Agent in LangChain do?

a) It manages user accounts and preferences.
b) It determines which tool or model to use based on the user's input.
c) It trains language models.
d) It is responsible for deploying the model.

Answer: b) It determines which tool or model to use based on the user's input.

7. Which of the following best describes "Prompts" in LangChain?

a) Predefined templates used to generate consistent input for models
b) The output generated by a language model
c) A way to store conversation data
d) A form of data validation

Answer: a) Predefined templates used to generate consistent input for models

```
%pip install -qU langchain-groq
```

```
%env GROQ_API_KEY=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
```

```
env: GROQ_API_KEY=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
```

```
import getpass
import os
if "GROQ_API_KEY" not in os.environ:
  os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq API key: ")


from langchain_groq import ChatGroq

llm = ChatGroq(
 model="mixtral-8x7b-32768",
 temperature=1,
 max_tokens=None,
 timeout=None,
 max_retries=2,
# other params...
)


speech = '''
My loving people,
We have been persuaded by some that are careful of our safety, to take heed how we commit our selves to armed multitudes, for fear of treach
Let tyrants fear. I have always so behaved myself that, under God, I have placed my chiefest strength and safeguard in the loyal hearts and
I know I have the body of a weak, feeble woman; but I have the heart and stomach of a king, and of a king of England too, and think foul sco
I know already, for your forwardness you have deserved rewards and crowns; and We do assure you on a word of a prince, they shall be duly pa
'''
messages = [
(
"system",
"You are a helpful assistant that summarize that summarize the input in 2 lines",
),
("human", speech),
]
ai_msg = llm.invoke(messages)
ai_msg
```

```
AIMessage(content='The speaker assures their people that they are not afraid of treachery from armed multitudes, and that they have
always found their greatest strength in the loyalty of their subjects. The speaker commends their subjects for their bravery and
promises to reward them appropriately.', additional_kwargs={}, response_metadata={'token_usage': {'completion_tokens': 55,
'prompt_tokens': 179, 'total_tokens': 234, 'completion_time': 0.084279945, 'prompt_time': 0.012747186, 'queue_time':
0.018521248999999997, 'total_time': 0.097027131}, 'model_name': 'mixtral-8x7b-32768', 'system_fingerprint': 'fp_c5f20b5bb1',
'finish_reason': 'stop', 'logprobs': None}, id='run-5d5db0a2-7be9-41ab-b8c4-31d49e957423-0', usage_metadata={'input_tokens': 179,
'output_tokens': 55, 'total_tokens': 234})
```

```
ai_msg.content
```

```
'The speaker assures their people that they are not afraid of treachery from armed multitudes, and that they have always found their gr
eatest strength in the loyalty of their subjects. The speaker commends their subjects for their bravery and promises to reward them app
```

```
speech
```

```
'\nMy loving people,\nWe have been persuaded by some that are careful of our safety, to take heed how we commit our selves to armed mul
titudes, for fear of treachery\nLet tyrants fear. I have always so behaved myself that, under God, I have placed my chiefest strength a
nd safeguard in the loyal hearts and goo\nI know I have the body of a weak, feeble woman; but I have the heart and stomach of a king, a
```

```
!pip install googletrans==3.1.0a0
```

```
from googletrans import Translator

def translate_text(text, dest='hi'):  # Default destination language is French
    """Translates text to the specified language.

    Args:
        text: The text to translate.
        dest: The destination language code (e.g., 'fr' for French, 'es' for Spanish).

    Returns:
        The translated text.
    """
```

```
    translator = Translator()
    translated = translator.translate(text, dest=dest)
    return translated.text

# Define the 'speech' variable within this cell before calling translate_text
speech = '''
My loving people,
We have been persuaded by some that are careful of our safety, to take heed how we commit our selves to armed multitudes, for fear of treache
Let tyrants fear. I have always so behaved myself that, under God, I have placed my chiefest strength and safeguard in the loyal hearts and g
I know I have the body of a weak, feeble woman; but I have the heart and stomach of a king, and of a king of England too, and think foul scor
I know already, for your forwardness you have deserved rewards and crowns; and We do assure you on a word of a prince, they shall be duly pai
'''

# Assuming 'speech' is defined in your previous cells, containing the text you want to translate
# Translate 'speech' to French
translated_speech = translate_text(speech, dest='hi')
print(translated_speech)
```

```
Requirement already satisfied: googletrans==3.1.0a0 in /usr/local/lib/python3.11/dist-packages (3.1.0a0)
Requirement already satisfied: httpx==0.13.3 in /usr/local/lib/python3.11/dist-packages (from googletrans==3.1.0a0) (0.13.3)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-packages (from httpx==0.13.3->googletrans==3.1.0a0) (2024.12.14
Requirement already satisfied: hstspreload in /usr/local/lib/python3.11/dist-packages (from httpx==0.13.3->googletrans==3.1.0a0) (2025.1
Requirement already satisfied: sniffio in /usr/local/lib/python3.11/dist-packages (from httpx==0.13.3->googletrans==3.1.0a0) (1.3.1)
Requirement already satisfied: chardet==3.* in /usr/local/lib/python3.11/dist-packages (from httpx==0.13.3->googletrans==3.1.0a0) (3.0.4
Requirement already satisfied: idna==2.* in /usr/local/lib/python3.11/dist-packages (from httpx==0.13.3->googletrans==3.1.0a0) (2.10)
Requirement already satisfied: rfc3986<2,>=1.3 in /usr/local/lib/python3.11/dist-packages (from httpx==0.13.3->googletrans==3.1.0a0) (1.
Requirement already satisfied: httpcore==0.9.* in /usr/local/lib/python3.11/dist-packages (from httpx==0.13.3->googletrans==3.1.0a0) (0.
Requirement already satisfied: h11<0.10,>=0.8 in /usr/local/lib/python3.11/dist-packages (from httpcore==0.9.*->httpx==0.13.3->googletra
Requirement already satisfied: h2==3.* in /usr/local/lib/python3.11/dist-packages (from httpcore==0.9.*->httpx==0.13.3->googletrans==3.1
Requirement already satisfied: hyperframe<6,>=5.2.0 in /usr/local/lib/python3.11/dist-packages (from h2==3.*->httpcore==0.9.*->httpx==0.
Requirement already satisfied: hpack<4,>=3.0 in /usr/local/lib/python3.11/dist-packages (from h2==3.*->httpcore==0.9.*->httpx==0.13.3->g
मेरे प्यार करने वाले लोग,
हम कुछ ऐसे लोगों द्वारा राजी कर लिए गए हैं जो हमारी सुरक्षा से सावधान हैं, यह ध्यान रखने के लिए कि हम विश्वास के डर से सशस्त्र मल्टीट्यूड के लिए कैसे प्रतिबद्ध हैं
अत्याचारियों को डर लगता है। मैंने हमेशा अपने आप को इतना व्यवहार किया है कि, भगवान के तहत, मैंने अपनी मुख्य शक्ति और वफादार दिलों और गू में सुरक्षा की है।
मुझे पता है कि मेरे पास एक कमजोर, कमजोर महिला का शरीर है; लेकिन मेरे पास एक राजा का दिल और पेट है, और इंग्लैंड के एक राजा का भी है, और सोचें
मैं पहले से ही जानता हूं, आपकी फॉरवर्डनेस के लिए आप पुरस्कार और मुकुट के हकदार हैं; और हम आपको एक राजकुमार के एक शब्द पर आश्वासन देते हैं, उन्हें विधिवत भु
```

Experiment No.–6				
Date of Performance:	24/2/25			
Date of Submission:	3/3/25			
Program Execution/ formation/ correction/ ethical practices (06)	Timely Submission (01)	Viva (03)	Experiment Total (10)	Sign with Date

6.1 Aim: Implementation of Generative AI Use Case- Summarize Dialogue using FLAN-T5

6.2 Course Outcome: 1.Successfully train these models using appropriate software tools and framework
2.Develop proficiency in programming languages and library commonly used in AI and ML

6.3 Learning Objectives:. Implementation of Generative AI Use Case- Summarize Dialogue

6.4 Requirement:.Google Colab

6.5 Related Theory:

FLAN-T5 is a large language model (LLM) developed by Google. It's based on the T5 (Text-To-Text Transfer Transformer) architecture and is further trained with a technique called "instruction fine-tuning." This extra training makes FLAN-T5 particularly good at following instructions and performing a wide variety of tasks in a zero-shot or few-shot manner.

Here's a breakdown of its key features and how it's used:

Key Features:

1. Instruction Fine-Tuning: Unlike the original T5, which was trained mainly on text-to-text tasks, FLAN-T5 was trained on a massive dataset of instructions and their corresponding responses. This enables it to understand and respond to instructions in natural language.
2. Zero-Shot and Few-Shot Learning: FLAN-T5 excels at zero-shot learning, meaning it can perform tasks it hasn't explicitly been trained on. It can also improve with just a few examples (few-shot learning). This makes it highly adaptable and versatile.
3. Text-To-Text Framework: FLAN-T5 uses a text-to-text approach, where any task is framed as converting input text into output text. This simplifies its usage across various applications, including translation, summarization, question answering, and more.
4. Open-Source: FLAN-T5 is open-source, available on platforms like Hugging Face, making it accessible for research and development.

6.6 Procedure:

Open google colab

1. Set up Kernel and Required Dependencies

1. **Check Instance Type:** Verify you are using the correct instance type (ml.m5.2xlarge). This is crucial for optimal performance. If it's incorrect, the code will stop and display an error.
2. **Install Packages:**
    - Install datasets, torch, torchdata, and transformers libraries using pip. These packages are essential for working with Hugging Face models and datasets.
3. **Load Dataset, Model, Tokenizer:**
    - Load the DialogSum dataset from Hugging Face using load_dataset.
    - Load the FLAN-T5 model using AutoModelForSeq2SeqLM.from_pretrained.
    - Load the tokenizer for the model using AutoTokenizer.from_pretrained.

2. Summarize Dialogue without Prompt Engineering

1. **Print Example Dialogues:** Print a couple of dialogues and their baseline human summaries to understand the data.
2. **Generate Summaries without Prompt Engineering:**
    - Iterate through example dialogues.
    - Tokenize the dialogue using the tokenizer.
    - Generate a summary using model.generate.
    - Decode the generated summary using the tokenizer.
    - Print the input dialogue, baseline summary, and the model-generated summary.

3. Summarize Dialogue with an Instruction Prompt

1. **Zero Shot Inference with an Instruction Prompt:**
    - Create a prompt by wrapping the dialogue with instructions like "Summarize the following conversation."
    - Tokenize the prompt.
    - Generate a summary using model.generate.
    - Decode the generated summary.
    - Print the prompt, baseline summary, and model-generated summary.
2. **Zero Shot Inference with the Prompt Template from FLAN-T5:**
    - Create a prompt using a pre-built template from FLAN-T5.
    - Follow the same steps as above (tokenize, generate, decode, print).

4. Summarize Dialogue with One Shot and Few Shot Inference

1. **One Shot Inference:**
    - Create a function make_prompt to generate a prompt with a full example (dialogue and summary).
    - Construct the prompt using this function.
    - Follow the same steps as above (tokenize, generate, decode, print).
2. **Few Shot Inference:**
    - Add more full examples to the prompt using make_prompt.

- Follow the same steps as above (tokenize, generate, decode, print).

5. Generative Configuration Parameters for Inference

1. **Experiment with GenerationConfig:**
   - Create a GenerationConfig object to control the generation parameters.
   - Modify parameters like max_new_tokens, do_sample, temperature, etc.
   - Follow the same steps as above (tokenize, generate, decode, print) to see the effect of the changes.

6.7 Program and Output:

6.9Conclusion

In this experiment we implemented of Generative AI Use Case- Summarize Dialogue

6.9 Questions

Question 1:

What does FLAN-T5 stand for?

(a) Fine-tuned Language Agnostic Network - Text-To-Text Transfer Transformer (b) Few-shot Learning Augmentation Network - Text-To-Speech Transfer Transformer (c) Fine-tuned Language Augmentation Network - Text-To-Text Transfer Transformer (d) Few-shot Learning Agnostic Network - Text-To-Speech Transfer Transformer

Answer: (c) Fine-tuned Language Augmentation Network - Text-To-Text Transfer Transformer

Question 2:

Which of the following is a key feature of FLAN-T5?

(a) It is pre-trained on a massive dataset of text and code. (b) It can be fine-tuned for a wide variety of tasks. (c) It is based on the T5 architecture. (d) All of the above.

Answer: (d) All of the above.

Question 3:

What is the primary use case for FLAN-T5?

(a) Image classification (b) Natural language processing (c) Speech recognition (d) Video generation

Answer: (b) Natural language processing

Question 4:

How does FLAN-T5 perform compared to other language models?

(a) It consistently outperforms other models on a variety of tasks. (b) It performs similarly to other models on most tasks. (c) It performs worse than other models on some tasks. (d) It is not possible to compare FLAN-T5 to other models.

Answer: (a) It consistently outperforms other models on a variety of tasks.

Question 5:

Which of the following is a limitation of FLAN-T5?

(a) It can be computationally expensive to train and use. (b) It can be difficult to fine-tune for specific tasks. (c) It can be biased towards the data it was trained on. (d) All of the above.

Answer: (d) All of the above.

```
import os

instance_type_expected = 'ml-m5-2xlarge'
# Instead of relying on HOSTNAME, let's try the EC2_INSTANCE_TYPE env variable
instance_type_current = os.environ.get('EC2_INSTANCE_TYPE')

# If EC2_INSTANCE_TYPE is not found, fallback to HOSTNAME
if instance_type_current is None:
    instance_type_current = os.environ.get('HOSTNAME')
    print("Warning: EC2_INSTANCE_TYPE not found, falling back to HOSTNAME")

print(f'Expected instance type: instance-datascience-{instance_type_expected}')
print(f'Currently chosen instance type: {instance_type_current}')

# The problem is likely that the current instance type does not EXACTLY match the expected instance type.
# We were checking if it was a substring before using "in". Now we will make it an exact check.
assert instance_type_expected == instance_type_current, f'ERROR. You selected the {instance_type_current} instance type. Please select {instar
print("Instance type has been chosen correctly.")
```

```
Warning: EC2_INSTANCE_TYPE not found, falling back to HOSTNAME
Expected instance type: instance-datascience-ml-m5-2xlarge
Currently chosen instance type: ced0d3647353
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
<ipython-input-4-fbb362c7f3f2> in <cell line: 0>()
     15 # The problem is likely that the current instance type does not EXACTLY match the expected instance type.
     16 # We were checking if it was a substring before using "in". Now we will make it an exact check.
---> 17 assert instance_type_expected == instance_type_current, f'ERROR. You selected the {instance_type_current} instance type. Please
select {instance_type_expected} instead as shown on the screenshot above'
     18 print("Instance type has been chosen correctly.")

AssertionError: ERROR. You selected the ced0d3647353 instance type. Please select ml-m5-2xlarge instead as shown on the screenshot
above
```

Next steps: Explain error

```
%pip install -U datasets==2.17.0

%pip install --upgrade pip
%pip install --disable-pip-version-check \
    torch==1.13.1 \
    torchdata==0.5.1 --quiet

%pip install \
    transformers==4.27.2 --quiet
```

```
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->datasets==2
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->datasets==2.17.0) (3.10
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.19.0->datasets==2.17.0)
```

```
torch 2.5.1+cu124 requires nvidia-cudnn-cu12==9.1.0.70; platform_system == "Linux" and platform_machine == "x86_64", but you have nvi
torch 2.5.1+cu124 requires nvidia-cufft-cu12==11.2.1.3; platform_system == "Linux" and platform_machine == "x86_64", but you have nvi
torch 2.5.1+cu124 requires nvidia-curand-cu12==10.3.5.147; platform_system == "Linux" and platform_machine == "x86_64", but you have n
torch 2.5.1+cu124 requires nvidia-cusolver-cu12==11.6.1.9; platform_system == "Linux" and platform_machine == "x86_64", but you have n
torch 2.5.1+cu124 requires nvidia-cusparse-cu12==12.3.1.170; platform_system == "Linux" and platform_machine == "x86_64", but you have
torch 2.5.1+cu124 requires nvidia-nvjitlink-cu12==12.4.127; platform_system == "Linux" and platform_machine == "x86_64", but you have
Successfully installed datasets-2.17.0 dill-0.3.8 fsspec-2023.10.0 multiprocess-0.70.16 pyarrow-hotfix-0.6 xxhash-3.5.0
Requirement already satisfied: pip in /usr/local/lib/python3.11/dist-packages (24.1.2)
Collecting pip
  Downloading pip-25.0.1-py3-none-any.whl.metadata (3.7 kB)
Downloading pip-25.0.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 16.5 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.1.2
    Uninstalling pip-24.1.2:
      Successfully uninstalled pip-24.1.2
Successfully installed pip-25.0.1
ERROR: Ignored the following yanked versions: 0.3.0a0
ERROR: Could not find a version that satisfies the requirement torchdata==0.5.1 (from versions: 0.3.0a1, 0.3.0, 0.6.0, 0.6.1, 0.7.0, 0
ERROR: No matching distribution found for torchdata==0.5.1
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.8/6.8 MB 51.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.8/7.8 MB 39.2 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the sourc
```

```
from datasets import load_dataset
from transformers import AutoModelForSeq2SeqLM
from transformers import AutoTokenizer
from transformers import GenerationConfig
```

```
huggingface_dataset_name = "knkarthick/dialogsum"

dataset = load_dataset(huggingface_dataset_name)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secre
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Downloading readme: 100%                                                  4.65k/4.65k [00:00<00:00, 118kB/s]
Downloading data: 100%                                                    11.3M/11.3M [00:00<00:00, 34.0MB/s]
Downloading data: 100%                                                    442k/442k [00:00<00:00, 3.71MB/s]
Downloading data: 100%                                                    1.35M/1.35M [00:00<00:00, 12.8MB/s]
Generating train split:        12460/0 [00:00<00:00, 35000.60 examples/s]
/usr/local/lib/python3.11/dist-packages/datasets/download/streaming_download_manager.py:784: FutureWarning: The 'verbose' keyword in pd.
  return pd.read_csv(xopen(filepath_or_buffer, "rb", download_config=download_config), **kwargs)
Generating validation split:        500/0 [00:00<00:00, 11040.95 examples/s]
/usr/local/lib/python3.11/dist-packages/datasets/download/streaming_download_manager.py:784: FutureWarning: The 'verbose' keyword in pd.
  return pd.read_csv(xopen(filepath_or_buffer, "rb", download_config=download_config), **kwargs)
Generating test split:        1500/0 [00:00<00:00, 22984.25 examples/s]
/usr/local/lib/python3.11/dist-packages/datasets/download/streaming_download_manager.py:784: FutureWarning: The 'verbose' keyword in pd.
  return pd.read_csv(xopen(filepath_or_buffer, "rb", download_config=download_config), **kwargs)
```

```
model_name='google/flan-t5-base'

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/file_download.py:795: FutureWarning: `resume_download` is deprecated and will be
  warnings.warn(
config.json: 100%                                                         1.40k/1.40k [00:00<00:00, 29.6kB/s]
/usr/local/lib/python3.11/dist-packages/huggingface_hub/file_download.py:795: FutureWarning: `resume_download` is deprecated and will be
  warnings.warn(
model.safetensors: 100%                                                   990M/990M [00:08<00:00, 111MB/s]
generation_config.json: 100%                                              147/147 [00:00<00:00, 11.8kB/s]
```

```
example_indices = [40, 200]

dash_line = '-'.join('' for x in range(100))
```

```
for i, index in enumerate(example_indices):
    print(dash_line)
    print('Example ', i + 1)
    print(dash_line)
    print('INPUT DIALOGUE:')
    print(dataset['test'][index]['dialogue'])
    print(dash_line)
    print('BASELINE HUMAN SUMMARY:')
    print(dataset['test'][index]['summary'])
    print(dash_line)
    print()
```

```
---------------------------------------------------------------------------------------------------
Example  1
---------------------------------------------------------------------------------------------------
INPUT DIALOGUE:
#Person1#: What time is it, Tom?
#Person2#: Just a minute. It's ten to nine by my watch.
#Person1#: Is it? I had no idea it was so late. I must be off now.
#Person2#: What's the hurry?
#Person1#: I must catch the nine-thirty train.
#Person2#: You've plenty of time yet. The railway station is very close. It won't take more than twenty minutes to get there.
---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# is in a hurry to catch a train. Tom tells #Person1# there is plenty of time.
---------------------------------------------------------------------------------------------------

---------------------------------------------------------------------------------------------------
Example  2
---------------------------------------------------------------------------------------------------
INPUT DIALOGUE:
#Person1#: Have you considered upgrading your system?
#Person2#: Yes, but I'm not sure what exactly I would need.
#Person1#: You could consider adding a painting program to your software. It would allow you to make up your own flyers and banners for
#Person2#: That would be a definite bonus.
#Person1#: You might also want to upgrade your hardware because it is pretty outdated now.
#Person2#: How can we do that?
#Person1#: You'd probably need a faster processor, to begin with. And you also need a more powerful hard disc, more memory and a faster
#Person2#: No.
#Person1#: Then you might want to add a CD-ROM drive too, because most new software programs are coming out on Cds.
#Person2#: That sounds great. Thanks.
---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# teaches #Person2# how to upgrade software and hardware in #Person2#'s system.
---------------------------------------------------------------------------------------------------
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
```

```
tokenizer_config.json: 100%  2.54k/2.54k [00:00<00:00, 96.1kB/s]
spiece.model: 100%  792k/792k [00:00<00:00, 21.7MB/s]
tokenizer.json: 100%  2.42M/2.42M [00:00<00:00, 35.9MB/s]
special_tokens_map.json: 100%  2.20k/2.20k [00:00<00:00, 55.4kB/s]
```

```
sentence = "What time is it, Tom?"

sentence_encoded = tokenizer(sentence, return_tensors='pt')

sentence_decoded = tokenizer.decode(
        sentence_encoded["input_ids"][0],
        skip_special_tokens=True
    )

print('ENCODED SENTENCE:')
print(sentence_encoded["input_ids"][0])
print('\nDECODED SENTENCE:')
print(sentence_decoded)
```

```
ENCODED SENTENCE:
tensor([ 363,   97,   19,   34,    6, 3059,   58,    1])

DECODED SENTENCE:
What time is it, Tom?
```

```
for i, index in enumerate(example_indices):
```

```
dialogue = dataset['test'][index]['dialogue']
summary = dataset['test'][index]['summary']

inputs = tokenizer(dialogue, return_tensors='pt')
output = tokenizer.decode(
    model.generate(
        inputs["input_ids"],
        max_new_tokens=50,
    )[0],
    skip_special_tokens=True
)

print(dash_line)
print('Example ', i + 1)
print(dash_line)
print(f'INPUT PROMPT:\n{dialogue}')
print(dash_line)
print(f'BASELINE HUMAN SUMMARY:\n{summary}')
print(dash_line)
print(f'MODEL GENERATION - WITHOUT PROMPT ENGINEERING:\n{output}\n')
```

```
---------------------------------------------------------------------------------------------------
Example  1
---------------------------------------------------------------------------------------------------
INPUT PROMPT:
#Person1#: What time is it, Tom?
#Person2#: Just a minute. It's ten to nine by my watch.
#Person1#: Is it? I had no idea it was so late. I must be off now.
#Person2#: What's the hurry?
#Person1#: I must catch the nine-thirty train.
#Person2#: You've plenty of time yet. The railway station is very close. It won't take more than twenty minutes to get there.
---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# is in a hurry to catch a train. Tom tells #Person1# there is plenty of time.
---------------------------------------------------------------------------------------------------
MODEL GENERATION - WITHOUT PROMPT ENGINEERING:
Person1: It's ten to nine.

---------------------------------------------------------------------------------------------------
Example  2
---------------------------------------------------------------------------------------------------
INPUT PROMPT:
#Person1#: Have you considered upgrading your system?
#Person2#: Yes, but I'm not sure what exactly I would need.
#Person1#: You could consider adding a painting program to your software. It would allow you to make up your own flyers and banners for
#Person2#: That would be a definite bonus.
#Person1#: You might also want to upgrade your hardware because it is pretty outdated now.
#Person2#: How can we do that?
#Person1#: You'd probably need a faster processor, to begin with. And you also need a more powerful hard disc, more memory and a faster
#Person2#: No.
#Person1#: Then you might want to add a CD-ROM drive too, because most new software programs are coming out on Cds.
#Person2#: That sounds great. Thanks.
---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# teaches #Person2# how to upgrade software and hardware in #Person2#'s system.
---------------------------------------------------------------------------------------------------
MODEL GENERATION - WITHOUT PROMPT ENGINEERING:
#Person1#: I'm thinking of upgrading my computer.
```

Experiment No.– 7				
Date of Performance:	24/2/25			
Date of Submission:	24/2/25			
Program Execution/ formation/ correction/ ethical practices (06)	Timely Submission (01)	Viva (03)	Experiment Total (10)	Sign with Date

7.1 Aim: Generative AI Use Case-Summarize Dialogue with prompt

7.2 Course Outcome: 1.Successfully train these models using appropriate software tools and framework
2.Develop proficiency in programming languages and library commonly used in AI and ML

7.3 Learning Objectives:. Implementation of Generative AI Use Case- Summarize Dialogue

6.4 Requirement:.Google Colab

7.5 Related Theory:

FLAN-T5 is a large language model (LLM) developed by Google. It's based on the T5 (Text-To-Text Transfer Transformer) architecture and is further trained with a technique called "instruction fine-tuning." This extra training makes FLAN-T5 particularly good at following instructions and performing a wide variety of tasks in a zero-shot or few-shot manner.

Here's a breakdown of its key features and how it's used:

Key Features:

1. Instruction Fine-Tuning: Unlike the original T5, which was trained mainly on text-to-text tasks, FLAN-T5 was trained on a massive dataset of instructions and their corresponding responses. This enables it to understand and respond to instructions in natural language.
2. Zero-Shot and Few-Shot Learning: FLAN-T5 excels at zero-shot learning, meaning it can perform tasks it hasn't explicitly been trained on. It can also improve with just a few examples (few-shot learning). This makes it highly adaptable and versatile.
3. Text-To-Text Framework: FLAN-T5 uses a text-to-text approach, where any task is framed as converting input text into output text. This simplifies its usage across various applications, including translation, summarization, question answering, and more.
4. Open-Source: FLAN-T5 is open-source, available on platforms like Hugging Face, making it accessible for research and development.

7.6 Procedure:

Open google colab

1. Set up Kernel and Required Dependencies

1. **Check Instance Type:** Verify you are using the correct instance type (ml.m5.2xlarge). This is crucial for optimal performance. If it's incorrect, the code will stop and display an error.
2. **Install Packages:**
   - Install datasets, torch, torchdata, and transformers libraries using pip. These packages are essential for working with Hugging Face models and datasets.
3. **Load Dataset, Model, Tokenizer:**
   - Load the DialogSum dataset from Hugging Face using load_dataset.
   - Load the FLAN-T5 model using AutoModelForSeq2SeqLM.from_pretrained.
   - Load the tokenizer for the model using AutoTokenizer.from_pretrained.

2. Summarize Dialogue without Prompt Engineering

1. **Print Example Dialogues:** Print a couple of dialogues and their baseline human summaries to understand the data.
2. **Generate Summaries without Prompt Engineering:**
   - Iterate through example dialogues.
   - Tokenize the dialogue using the tokenizer.
   - Generate a summary using model.generate.
   - Decode the generated summary using the tokenizer.
   - Print the input dialogue, baseline summary, and the model-generated summary.

3. Summarize Dialogue with an Instruction Prompt

1. **Zero Shot Inference with an Instruction Prompt:**
   - Create a prompt by wrapping the dialogue with instructions like "Summarize the following conversation."
   - Tokenize the prompt.
   - Generate a summary using model.generate.
   - Decode the generated summary.
   - Print the prompt, baseline summary, and model-generated summary.
2. **Zero Shot Inference with the Prompt Template from FLAN-T5:**
   - Create a prompt using a pre-built template from FLAN-T5.
   - Follow the same steps as above (tokenize, generate, decode, print).

4. Summarize Dialogue with One Shot and Few Shot Inference

1. **One Shot Inference:**
   - Create a function make_prompt to generate a prompt with a full example (dialogue and summary).
   - Construct the prompt using this function.
   - Follow the same steps as above (tokenize, generate, decode, print).
2. **Few Shot Inference:**
   - Add more full examples to the prompt using make_prompt.

- Follow the same steps as above (tokenize, generate, decode, print).

5. Generative Configuration Parameters for Inference

1. **Experiment with GenerationConfig:**
   - Create a GenerationConfig object to control the generation parameters.
   - Modify parameters like max_new_tokens, do_sample, temperature, etc.
   - Follow the same steps as above (tokenize, generate, decode, print) to see the effect of the changes.

7.7 Procedure And Code :

The documents are attached in next file.

7.8Conclusion

In this experiment we implemented of Generative AI Use Case- Summarize Dialogue

7.9 Questions

Question 1:

What does FLAN-T5 stand for?

(a) Fine-tuned Language Agnostic Network - Text-To-Text Transfer Transformer (b) Few-shot Learning Augmentation Network - Text-To-Speech Transfer Transformer (c) Fine-tuned Language Augmentation Network - Text-To-Text Transfer Transformer (d) Few-shot Learning Agnostic Network - Text-To-Speech Transfer Transformer

Answer: (c) Fine-tuned Language Augmentation Network - Text-To-Text Transfer Transformer

Question 2:

Which of the following is a key feature of FLAN-T5?

(a) It is pre-trained on a massive dataset of text and code. (b) It can be fine-tuned for a wide variety of tasks. (c) It is based on the T5 architecture. (d) All of the above.

Answer: (d) All of the above.

Question 3:

What is the primary use case for FLAN-T5?

(a) Image classification (b) Natural language processing (c) Speech recognition (d) Video generation

Answer: (b) Natural language processing

Question 4:

How does FLAN-T5 perform compared to other language models?

(a) It consistently outperforms other models on a variety of tasks. (b) It performs similarly to other models on most tasks. (c) It performs worse than other models on some tasks. (d) It is not possible to compare FLAN-T5 to other models.

Answer: (a) It consistently outperforms other models on a variety of tasks.

Question 5:

Which of the following is a limitation of FLAN-T5?

(a) It can be computationally expensive to train and use. (b) It can be difficult to fine-tune for specific tasks. (c) It can be biased towards the data it was trained on. (d) All of the above.

Answer: (d) All of the above.

```
for i, index in enumerate(example_indices):
    dialogue = dataset['test'][index]['dialogue']
    summary = dataset['test'][index]['summary']

    prompt = f"""
Summarize the following conversation.

{dialogue}

Summary:
    """

    # Input constructed prompt instead of the dialogue.
    inputs = tokenizer(prompt, return_tensors='pt')
    output = tokenizer.decode(
        model.generate(
            inputs["input_ids"],
            max_new_tokens=50,
        )[0],
        skip_special_tokens=True
    )

    print(dash_line)
    print('Example ', i + 1)
    print(dash_line)
    print(f'INPUT PROMPT:\n{prompt}')
    print(dash_line)
    print(f'BASELINE HUMAN SUMMARY:\n{summary}')
    print(dash_line)
    print(f'MODEL GENERATION - ZERO SHOT:\n{output}\n')
```

```
---------------------------------------------------------------------------------------------------
Example  1
---------------------------------------------------------------------------------------------------
INPUT PROMPT:

Summarize the following conversation.

#Person1#: What time is it, Tom?
#Person2#: Just a minute. It's ten to nine by my watch.
#Person1#: Is it? I had no idea it was so late. I must be off now.
#Person2#: What's the hurry?
#Person1#: I must catch the nine-thirty train.
#Person2#: You've plenty of time yet. The railway station is very close. It won't take more than twenty minutes to get there.

Summary:

---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# is in a hurry to catch a train. Tom tells #Person1# there is plenty of time.
---------------------------------------------------------------------------------------------------
MODEL GENERATION - ZERO SHOT:
The train is about to leave.

---------------------------------------------------------------------------------------------------
Example  2
---------------------------------------------------------------------------------------------------
INPUT PROMPT:

Summarize the following conversation.

#Person1#: Have you considered upgrading your system?
#Person2#: Yes, but I'm not sure what exactly I would need.
#Person1#: You could consider adding a painting program to your software. It would allow you to make up your own flyers and banners
#Person2#: That would be a definite bonus.
#Person1#: You might also want to upgrade your hardware because it is pretty outdated now.
#Person2#: How can we do that?
#Person1#: You'd probably need a faster processor, to begin with. And you also need a more powerful hard disc, more memory and a fas
#Person2#: No.
#Person1#: Then you might want to add a CD-ROM drive too, because most new software programs are coming out on Cds.
#Person2#: That sounds great. Thanks.

Summary:

---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# teaches #Person2# how to upgrade software and hardware in #Person2#'s system.
---------------------------------------------------------------------------------------------------
MODEL GENERATION - ZERO SHOT:
#Person1#: I'm thinking of upgrading my computer.
```

```
for i, index in enumerate(example_indices):
    dialogue = dataset['test'][index]['dialogue']
```

```
    summary = dataset['test'][index]['summary']

    prompt = f"""
Dialogue:

{dialogue}

What was going on?
"""

    inputs = tokenizer(prompt, return_tensors='pt')
    output = tokenizer.decode(
        model.generate(
            inputs["input_ids"],
            max_new_tokens=50,
        )[0],
        skip_special_tokens=True
    )

    print(dash_line)
    print('Example ', i + 1)
    print(dash_line)
    print(f'INPUT PROMPT:\n{prompt}')
    print(dash_line)
    print(f'BASELINE HUMAN SUMMARY:\n{summary}\n')
    print(dash_line)
    print(f'MODEL GENERATION - ZERO SHOT:\n{output}\n')
```

```
---------------------------------------------------------------------------------------------------
Example  1
---------------------------------------------------------------------------------------------------
INPUT PROMPT:

Dialogue:

#Person1#: What time is it, Tom?
#Person2#: Just a minute. It's ten to nine by my watch.
#Person1#: Is it? I had no idea it was so late. I must be off now.
#Person2#: What's the hurry?
#Person1#: I must catch the nine-thirty train.
#Person2#: You've plenty of time yet. The railway station is very close. It won't take more than twenty minutes to get there.

What was going on?

---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# is in a hurry to catch a train. Tom tells #Person1# there is plenty of time.

---------------------------------------------------------------------------------------------------
MODEL GENERATION - ZERO SHOT:
Tom is late for the train.

---------------------------------------------------------------------------------------------------
Example  2
---------------------------------------------------------------------------------------------------
INPUT PROMPT:

Dialogue:

#Person1#: Have you considered upgrading your system?
#Person2#: Yes, but I'm not sure what exactly I would need.
#Person1#: You could consider adding a painting program to your software. It would allow you to make up your own flyers and banners
#Person2#: That would be a definite bonus.
#Person1#: You might also want to upgrade your hardware because it is pretty outdated now.
#Person2#: How can we do that?
#Person1#: You'd probably need a faster processor, to begin with. And you also need a more powerful hard disc, more memory and a fas
#Person2#: No.
#Person1#: Then you might want to add a CD-ROM drive too, because most new software programs are coming out on Cds.
#Person2#: That sounds great. Thanks.

What was going on?

---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# teaches #Person2# how to upgrade software and hardware in #Person2#'s system.

---------------------------------------------------------------------------------------------------
MODEL GENERATION - ZERO SHOT:
#Person1#: You could add a painting program to your software. #Person2#: That would be a bonus. #Person1#: You might also want to up
```

Experiment No -8				
Date of Performance:	24/2/25			
Date of Submission:	24/2/25			
Program Execution/ formation/ correction/ ethical practices (06)	Timely Submission (01)	Viva (03)	Experiment Total (10)	Sign with Date

8.1 Aim: Use Case- Summarize Dialogue with one and few shots

8.2 Course Outcome: 1.Successfully train these models using appropriate software tools and framework
2.Develop proficiency in programming languages and library commonly used in AI and ML

8.3 Learning Objectives:. Implementation of Generative AI Use Case- Summarize Dialogue

8.4 Requirement : Google collab

8.5 Related Theory:

FLAN-T5 is a large language model (LLM) developed by Google. It's based on the T5 (Text-To-Text Transfer Transformer) architecture and is further trained with a technique called "instruction fine-tuning." This extra training makes FLAN-T5 particularly good at following instructions and performing a wide variety of tasks in a zero-shot or few-shot manner.

Here's a breakdown of its key features and how it's used:

Key Features:

1. Instruction Fine-Tuning: Unlike the original T5, which was trained mainly on text-to-text tasks, FLAN-T5 was trained on a massive dataset of instructions and their corresponding responses. This enables it to understand and respond to instructions in natural language.
2. Zero-Shot and Few-Shot Learning: FLAN-T5 excels at zero-shot learning, meaning it can perform tasks it hasn't explicitly been trained on. It can also improve with just a few examples (few-shot learning). This makes it highly adaptable and versatile.
3. Text-To-Text Framework: FLAN-T5 uses a text-to-text approach, where any task is framed as converting input text into output text. This simplifies its usage across various applications, including translation, summarization, question answering, and more.
4. Open-Source: FLAN-T5 is open-source, available on platforms like Hugging Face, making it accessible for research and development.

8.6 Procedure:

Open google colab

1. Set up Kernel and Required Dependencies

1. **Check Instance Type:** Verify you are using the correct instance type (ml.m5.2xlarge). This is crucial for optimal performance. If it's incorrect, the code will stop and display an error.
2. **Install Packages:**
   - Install datasets, torch, torchdata, and transformers libraries using pip. These packages are essential for working with Hugging Face models and datasets.
3. **Load Dataset, Model, Tokenizer:**
   - Load the DialogSum dataset from Hugging Face using load_dataset.
   - Load the FLAN-T5 model using AutoModelForSeq2SeqLM.from_pretrained.
   - Load the tokenizer for the model using AutoTokenizer.from_pretrained.

2. Summarize Dialogue without Prompt Engineering

1. **Print Example Dialogues:** Print a couple of dialogues and their baseline human summaries to understand the data.
2. **Generate Summaries without Prompt Engineering:**
   - Iterate through example dialogues.
   - Tokenize the dialogue using the tokenizer.
   - Generate a summary using model.generate.
   - Decode the generated summary using the tokenizer.
   - Print the input dialogue, baseline summary, and the model-generated summary.

3. Summarize Dialogue with an Instruction Prompt

1. **Zero Shot Inference with an Instruction Prompt:**
   - Create a prompt by wrapping the dialogue with instructions like "Summarize the following conversation."
   - Tokenize the prompt.
   - Generate a summary using model.generate.
   - Decode the generated summary.
   - Print the prompt, baseline summary, and model-generated summary.
2. **Zero Shot Inference with the Prompt Template from FLAN-T5:**
   - Create a prompt using a pre-built template from FLAN-T5.
   - Follow the same steps as above (tokenize, generate, decode, print).

4. Summarize Dialogue with One Shot and Few Shot Inference

1. **One Shot Inference:**
   - Create a function make_prompt to generate a prompt with a full example (dialogue and summary).
   - Construct the prompt using this function.
   - Follow the same steps as above (tokenize, generate, decode, print).
2. **Few Shot Inference:**
   - Add more full examples to the prompt using make_prompt.

- Follow the same steps as above (tokenize, generate, decode, print).

5. Generative Configuration Parameters for Inference

1. **Experiment with GenerationConfig:**
   - Create a GenerationConfig object to control the generation parameters.
   - Modify parameters like max_new_tokens, do_sample, temperature, etc.
   - Follow the same steps as above (tokenize, generate, decode, print) to see the effect of the changes.

8.7 Procedure And Code :

The documents are attached in next file.

8.8Conclusion

In this experiment we implemented of Generative AI Use Case- Summarize Dialogue

8.9 Questions

Question 1:

What does FLAN-T5 stand for?

(a) Fine-tuned Language Agnostic Network - Text-To-Text Transfer Transformer (b) Few-shot Learning Augmentation Network - Text-To-Speech Transfer Transformer (c) Fine-tuned Language Augmentation Network - Text-To-Text Transfer Transformer (d) Few-shot Learning Agnostic Network - Text-To-Speech Transfer Transformer

Answer: (c) Fine-tuned Language Augmentation Network - Text-To-Text Transfer Transformer

Question 2:

Which of the following is a key feature of FLAN-T5?

(a) It is pre-trained on a massive dataset of text and code. (b) It can be fine-tuned for a wide variety of tasks. (c) It is based on the T5 architecture. (d) All of the above.

Answer: (d) All of the above.

Question 3:

What is the primary use case for FLAN-T5?

(a) Image classification (b) Natural language processing (c) Speech recognition (d) Video generation

Answer: (b) Natural language processing

Question 4:

How does FLAN-T5 perform compared to other language models?

(a) It consistently outperforms other models on a variety of tasks. (b) It performs similarly to other models on most tasks. (c) It performs worse than other models on some tasks. (d) It is not possible to compare FLAN-T5 to other models.

Answer: (a) It consistently outperforms other models on a variety of tasks.

Question 5:

Which of the following is a limitation of FLAN-T5?

(a) It can be computationally expensive to train and use. (b) It can be difficult to fine-tune for specific tasks. (c) It can be biased towards the data it was trained on. (d) All of the above.

Answer: (d) All of the above.

```
Start coding or generate with AI.
```

```
def make_prompt(example_indices_full, example_index_to_summarize):
    prompt = ''
    for index in example_indices_full:
        dialogue = dataset['test'][index]['dialogue']
        summary = dataset['test'][index]['summary']

        # The stop sequence '{summary}\n\n\n' is important for FLAN-T5. Other models may have their own preferred stop sequence.
        prompt += f"""
Dialogue:

{dialogue}

What was going on?
{summary}


"""

    dialogue = dataset['test'][example_index_to_summarize]['dialogue']

    prompt += f"""
Dialogue:

{dialogue}

What was going on?
"""

    return prompt


example_indices_full = [40]
example_index_to_summarize = 200

one_shot_prompt = make_prompt(example_indices_full, example_index_to_summarize)

print(one_shot_prompt)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-2-1443ab3f328f> in <cell line: 0>()
      2 example_index_to_summarize = 200
      3
----> 4 one_shot_prompt = make_prompt(example_indices_full, example_index_to_summarize)
      5
      6 print(one_shot_prompt)

<ipython-input-1-28eac7059d5e> in make_prompt(example_indices_full, example_index_to_summarize)
      2     prompt = ''
      3     for index in example_indices_full:
----> 4         dialogue = dataset['test'][index]['dialogue']
      5         summary = dataset['test'][index]['summary']
      6

NameError: name 'dataset' is not defined
```

Next steps: Explain error

```
example_indices_full = [40, 80, 120]
example_index_to_summarize = 200

few_shot_prompt = make_prompt(example_indices_full, example_index_to_summarize)

print(few_shot_prompt)
```

```
Dialogue:

#Person1#: What time is it, Tom?
#Person2#: Just a minute. It's ten to nine by my watch.
#Person1#: Is it? I had no idea it was so late. I must be off now.
#Person2#: What's the hurry?
#Person1#: I must catch the nine-thirty train.
#Person2#: You've plenty of time yet. The railway station is very close. It won't take more than twenty minutes to get there.

What was going on?
#Person1# is in a hurry to catch a train. Tom tells #Person1# there is plenty of time.
```

```
Dialogue:

#Person1#: May, do you mind helping me prepare for the picnic?
#Person2#: Sure. Have you checked the weather report?
#Person1#: Yes. It says it will be sunny all day. No sign of rain at all. This is your father's favorite sausage. Sandwiches for
#Person2#: No, thanks Mom. I'd like some toast and chicken wings.
#Person1#: Okay. Please take some fruit salad and crackers for me.
#Person2#: Done. Oh, don't forget to take napkins disposable plates, cups and picnic blanket.
#Person1#: All set. May, can you help me take all these things to the living room?
#Person2#: Yes, madam.
#Person1#: Ask Daniel to give you a hand?
#Person2#: No, mom, I can manage it by myself. His help just causes more trouble.

What was going on?
Mom asks May to help to prepare for the picnic and May agrees.



Dialogue:

#Person1#: Hello, I bought the pendant in your shop, just before.
#Person2#: Yes. Thank you very much.
#Person1#: Now I come back to the hotel and try to show it to my friend, the pendant is broken, I'm afraid.
#Person2#: Oh, is it?
#Person1#: Would you change it to a new one?
#Person2#: Yes, certainly. You have the receipt?
#Person1#: Yes, I do.
#Person2#: Then would you kindly come to our shop with the receipt by 10 o'clock? We will replace it.
#Person1#: Thank you so much.

What was going on?
#Person1# wants to change the broken pendant in #Person2#'s shop.



Dialogue:

#Person1#: Have you considered upgrading your system?
#Person2#: Yes, but I'm not sure what exactly I would need.
#Person1#: You could consider adding a painting program to your software. It would allow you to make up your own flyers and banne
#Person2#: That would be a definite bonus.
#Person1#: You might also want to upgrade your hardware because it is pretty outdated now.
```

```
summary = dataset['test'][example_index_to_summarize]['summary']

inputs = tokenizer(few_shot_prompt, return_tensors='pt')
output = tokenizer.decode(
    model.generate(
        inputs["input_ids"],
        max_new_tokens=50,
    )[0],
    skip_special_tokens=True
)

print(dash_line)
print(f'BASELINE HUMAN SUMMARY:\n{summary}\n')
print(dash_line)
print(f'MODEL GENERATION - FEW SHOT:\n{output}')
```

```
Token indices sequence length is longer than the specified maximum sequence length for this model (819 > 512). Running this sequence
---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# teaches #Person2# how to upgrade software and hardware in #Person2#'s system.

---------------------------------------------------------------------------------------------------
MODEL GENERATION - FEW SHOT:
#Person1 wants to upgrade his system. #Person2 wants to add a painting program to his software. #Person1 wants to upgrade his hardwa
```

```
generation_config = GenerationConfig(max_new_tokens=50)
# generation_config = GenerationConfig(max_new_tokens=10)
# generation_config = GenerationConfig(max_new_tokens=50, do_sample=True, temperature=0.1)
# generation_config = GenerationConfig(max_new_tokens=50, do_sample=True, temperature=0.5)
# generation_config = GenerationConfig(max_new_tokens=50, do_sample=True, temperature=1.0)

inputs = tokenizer(few_shot_prompt, return_tensors='pt')
output = tokenizer.decode(
    model.generate(
        inputs["input_ids"],
        generation_config=generation_config,
    )[0],
    skip_special_tokens=True
)

print(dash_line)
```

```
print(f'MODEL GENERATION - FEW SHOT:\n{output}')
print(dash_line)
print(f'BASELINE HUMAN SUMMARY:\n{summary}\n')
```

```
---------------------------------------------------------------------------------------------------
MODEL GENERATION - FEW SHOT:
#Person1 wants to upgrade his system. #Person2 wants to add a painting program to his software. #Person1 wants to upgrade his hardwa
---------------------------------------------------------------------------------------------------
BASELINE HUMAN SUMMARY:
#Person1# teaches #Person2# how to upgrade software and hardware in #Person2#'s system.
```